

NEMO: Execution-Aware Optimization Modeling via Autonomous Coding Agents

Yang Song^{*1} Anoushka Vyas^{*1} Zirui Wei¹ Sina Khoshfetrat Pakazad¹ Henrik Ohlsson¹ Graham Neubig²

Abstract

We present **NEMO**, a system that translates Natural-language descriptions of decision problems into formal Executable Mathematical Optimization implementations using autonomous coding agents (ACAs). Existing approaches rely on specialized large language models (LLMs) or bespoke task-specific agents that are often brittle and frequently generate syntactically invalid or non-executable code. NEMO instead treats ACAs as a first-class abstraction analogous to API-based interaction with LLMs; their sandboxed execution guarantees code is executable by construction and supports automated validation and repair. We introduce novel coordination patterns including asymmetric validation loops between independently generated optimizer and simulator implementations, external memory for experience reuse, and robustness enhancements via minimum Bayes risk (MBR) decoding and self-consistency. Across nine established optimization benchmarks, NEMO achieves state-of-the-art performance on the majority of tasks with substantial margins on several datasets (Figure 1), demonstrating the power of execution-aware agentic architectures for automated optimization modeling.

1. Introduction

Optimization-based decision problems arise across a wide range of domains, including supply chain management, resource allocation, portfolio construction, and energy systems planning (Singh, 2012; Saghafian et al., 2015; Shakoor et al., 2016; Cornuéjols et al., 2018; Antoniou & Lu, 2021;

^{*}Equal contribution ¹C3 AI, Redwood City, CA, USA
²Language Technologies Institute, School of Computer Science, Carnegie Mellon University, Pittsburgh, PA, USA. Correspondence to: Sina Khoshfetrat Pakazad <sina.pakazad@c3.ai>.

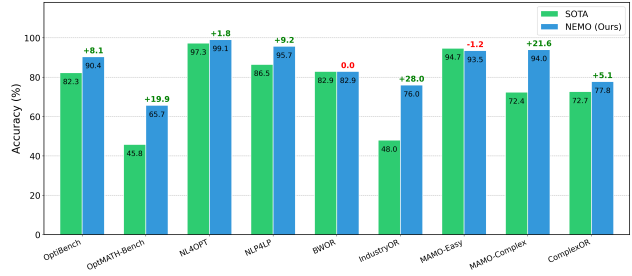


Figure 1. Accuracy comparison between NEMO and the reported SOTA results across nine optimization benchmarks. NEMO outperforms prior SOTA on eight of nine benchmarks, with absolute gains of up to 28 percentage points. Full results are reported in Table 1.

DeCroix et al., 2021). These problems often involve thousands of variables, complex constraints, and domain-specific structure, requiring careful formulation and expert knowledge to solve reliably. As a result, developing effective optimization solutions remains a labor-intensive process that depends on close collaboration between end-users, domain experts, and highly skilled operations research practitioners.

This development process is inherently iterative. Beyond initial formulation, optimization models must be repeatedly revised as business objectives evolve, operational constraints change, and new data becomes available. These feedback loops, spanning problem specification, solver selection, formulation, implementation, and evaluation, are costly and slow. This in turn creates a significant bottleneck that limits access to optimization-driven decision-making. Consequently, the value of optimization technologies remains largely confined to organizations with sustained access to specialized expertise.

At a high level, this workflow consists of three recurring steps, namely, identifying the key components of the decision process (e.g., decision variables, constraints, objectives, and exogenous factors), selecting appropriate solution techniques, and formulating and implementing the corresponding optimization model. The resulting solutions are then evaluated by domain experts, often by mentally simulating system behavior and assessing feasibility and plausibility,

before further refinement (a simulator-optimizer feedback loop).

Recent advances in LLMs offer a promising avenue to lower this barrier by automating parts of the optimization modeling pipeline. Prior work has explored both training-based approaches, which fine-tune LLMs for optimization tasks (Huang et al., 2025a; Chen et al., 2025; Jiang et al., 2025), and agent-based frameworks that orchestrate general-purpose LLMs through specialized components (Xiao et al., 2024; Thind et al., 2025; Ahmaditeshnizi et al., 2024; Zhang et al., 2025a). While these methods have demonstrated encouraging progress, they suffer from fundamental limitations. Because they rely primarily on direct code generation without execution-aware validation (or ad-hoc versions of execution-based debugging), they are often brittle, frequently producing syntactically invalid or non-executable implementations. More importantly (even when relying on execution-aware debugging), they lack the sophistication and ability to instantiate the simulator-optimizer feedback loops that practitioners rely on to uncover logical inconsistencies and modeling errors, as doing so requires generating, executing and refining both simulation and optimization code iteratively and collaboratively.

In this paper, we propose a system (NEMO) that combines direct usage of LLMs with remote interaction with ACAs to enable reliable, execution-aware translation of natural-language decision descriptions into optimization models. Our design is explicitly inspired by the human-in-the-loop workflow used by optimization practitioners. By leveraging ACAs that are equipped with sandboxed execution environments, the system ensures that generated implementations are executable by construction and can be systematically validated and refined.

We evaluate NEMO in fully autonomous mode across nine established optimization benchmarks. Despite relying only on widely available and general-purpose LLMs that predate recent frontier releases, NEMO achieves state-of-the-art performance on eight benchmarks and competitive results on the remaining one, see Figure 1 and Table 1. These results demonstrate that execution-aware, agentic architectures can substantially improve the robustness and reliability of language-driven decision optimization.

2. ACAs for Optimization Modeling

A central abstraction in NEMO is remote interaction with ACAs, acting as execution-capable counterparts to LLMs. Unlike standard LLM calls that produce text-only outputs, ACAs operate within sandboxed execution environments that support code generation, execution, inspection, and iterative modification, enabling execution-aware validation.

The system interacts with an ACA through a remote inter-

face that submits task specifications, comprising natural-language instructions, structured problem descriptions, and references to existing artifacts, and receives executable code, execution traces, and results in return. While ACA interactions are stateless at the interface level, they may reference persistent artifacts and memory managed by the system, allowing asynchronous coordination while preserving isolation and reproducibility. In our implementation, we instantiate this abstraction using *OpenHands* (Wang et al., 2025), though the framework is agnostic to the underlying ACA platform.

2.1. Opportunities with ACAs

The use of ACAs as a first-class abstraction for optimization modeling introduces distinct advantages over approaches based on specialized LLMs or bespoke task-specific agents. We refer to this as a *Workspace-State* paradigm: agents co-inhabit a persistent sandboxed environment and interact via executable artifacts, in contrast to the message-passing paradigm of prior systems that exchange structured text through rigid I/O schemas. This design yields three key capabilities. First, the ACA acts as an architect: generated code is executable by construction, with execution, debugging, and recovery handled natively by the ACA itself, and each agent constructs a self-contained package managing dependencies, utilities, and unit tests, eliminating the need for a separate critic to interpret errors. Second, a shared workspace allows independent ACAs instantiated for different roles (e.g., simulation and optimization) to co-reside, enabling direct module imports and cross-validation without schema negotiation. Third, executable memory incorporates prior experience and exemplars directly into the ACA codebase as runnable files rather than static prompt text, allowing the agent to execute or import from them. This clean separation between high-level decision reasoning and low-level code execution underpins the asymmetric validation and coordination mechanisms described in Section 3.7.

2.2. Challenges with ACAs

While ACAs provide significant advantages, this paradigm introduces unique challenges that motivate our technical contributions. ACAs exhibit inherent non-determinism in both code structure and execution outcomes, manifesting as differences in variable naming, constraint formulation, solver configuration, and numerical precision. Moreover, while sandboxed execution guarantees syntactic validity, it does not ensure semantic correctness, generated code may execute successfully yet encode an incorrect formulation or violate problem constraints. Without ground-truth solutions, validating semantic correctness becomes particularly challenging. These challenges motivate the systematic mechanisms introduced in Section 3.

3. Methodology

3.1. Method Overview

NEMO leverages the benefits of ACAs and addresses the challenges identified in Section 2.2 through a coordinated multi-component architecture (Figure 2) consisting of four primary modules: a decision process extractor that converts natural-language descriptions into structured representations using consensus-based decoding (Section 3.3); a simulator that constructs an executable model to evaluate feasibility and objective values (Section 3.5); a solver recommender that selects appropriate optimization backends (Section 3.4); and an optimizer that generates and refines executable solver code using self-consistency mechanisms (Section 3.6).

The system exploits the asymmetry between simulation and optimization complexity through a validation loop in which the simulator serves as a fixed executable reference for validating optimizer outputs (Section 3.7). To further improve robustness, selected modules employ diversity-aware memory retrieval for few-shot learning (Section 3.2), MBR decoding to stabilize extractions (Section 3.3.1), and self-consistency aggregation to ensure solution reliability (Section 3.6).

3.2. Memory for Few-shot Learning

The effectiveness of in-context learning and few-shot examples for improving the performance of LLM-based systems is well established (Brown et al., 2020; Li & Liang, 2021; Schick & Schütze, 2021; OpenAI, 2023; Liu et al., 2023). Motivated by this, we equip both the decision process extractor and the optimizer with access to a shared memory that enables reuse of prior problem-solving experience beyond standard prompt-based conditioning.

We construct this memory using a subset of the OptMATH (Lu et al., 2025) training dataset, which provides diverse and structured examples of optimization problems. Each sample i in the dataset is represented as a triplet (D_i, I_i, C_i) , where D_i denotes a natural-language problem description, I_i the corresponding mathematical formulation, and C_i the associated optimization code. From this dataset, we select a memory bank of 3,000 samples chosen to maximize coverage across 15 distinct problem types including knapsack, scheduling, routing, and facility location problems (see Appendix B.4 for the complete taxonomy).

To enable efficient retrieval, we embed all problem descriptions D_i into a dense vector space and construct a vector-store over these embeddings. Given a new problem description D , we first retrieve a candidate pool $\mathcal{M}$ of the top- N examples based on cosine similarity,

$$\text{sim}(D, D_i) = \cos(\text{embed}(D), \text{embed}(D_i)),$$

where $\text{embed}(\cdot)$ denotes a dense embedding function. We

restrict retrieval to problem descriptions, as this is the only modality available at inference time prior to formulation, and empirical similarity in this space provides sufficient signal for identifying structurally related optimization problems.

From the candidate pool $\mathcal{M}$, we select a subset $\mathcal{M}^*$ of k samples using a greedy strategy that balances relevance and diversity. We initialize $\mathcal{M}^*$ with the single candidate in $\mathcal{M}$ most similar to D . We then iteratively add the candidate $c \in \mathcal{M} \setminus \mathcal{M}^*$ that maximizes the following scoring function until $|\mathcal{M}^*| = k$,

$$\text{score}(c) = \text{sim}(D, c) - \lambda \cdot \frac{1}{|\mathcal{M}^*|} \sum_{m \in \mathcal{M}^*} \text{sim}(c, m).$$

The second term penalizes redundancy by measuring the average similarity between the candidate and the examples already selected in $\mathcal{M}^*$. This formulation keeps both similarity and diversity terms bounded in $[0, 1]$, ensuring consistent behavior of the trade-off parameter λ across retrieval steps.

Although all candidates in $\mathcal{M}$ are highly similar to the target problem, incorporating diversity mitigates bias toward frequently occurring patterns and guards against collapse to near-duplicate examples. To validate the impact of this parameter, we provide an ablation study over λ in Appendix B.4.1. Retrieved samples are used as soft guidance rather than hard constraints. Their associated formulations I_i are provided to the decision process extractor, while code artifacts C_i are supplied to the optimizer. Notably, we employ different mechanisms for incorporating retrieved examples into the decision process extractor and the optimizer, respectively; these module-specific integration strategies are described in Sections 3.3 and 3.6.

3.3. Decision Process Extractor

The decision process extractor is responsible for translating a natural-language description of a decision problem into a structured, machine or human interpretable representation. To this end, we leverage a carefully prompted reasoning LLM to extract the key components that define a decision process. Inspired by the decision modeling framework of Powell (2022), given a natural-language description D and a set of retrieved examples $\mathcal{M}^*$, the extractor produces a structured representation $\mathcal{P}$ consisting of the following elements: decision variables, exogenous variables and uncertainties, state variables, transition dynamics, objective function, and constraints. In addition to the structural components, $\mathcal{P}$ also contains inferred default values for exogenous variables and other parameters that specify the objective function and constraints, extracted from D . Formally, this extraction can be expressed as $\mathcal{E} : (D, \mathcal{M}^*) \rightarrow \mathcal{P}$.

A central challenge in using reasoning LLMs for decision

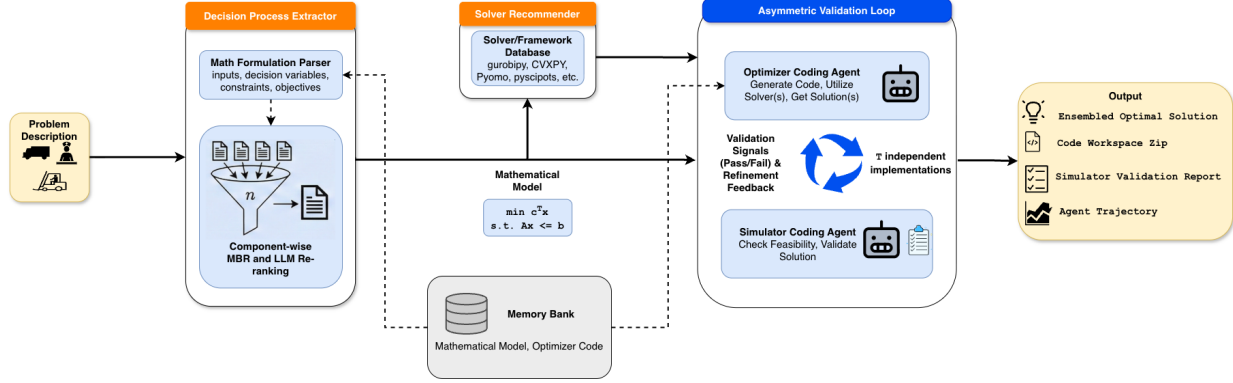


Figure 2. Overview of NEMO. Natural language descriptions are translated into formal mathematical models via component-wise MBR decoding. These models drive an asymmetric validation loop between independent optimizer and simulator agents, where the simulator detects feasibility errors and guides iterative refinement. The system leverages external memory and solver recommendations to produce validated, executable optimization code.

process extraction is their inherent non-determinism. Even when conditioned on identical inputs, such models can produce variable outputs in terms of structure, formatting, and interpretation of extracted components. Because the decision process extractor operates at the upstream end of the system pipeline, variability at this stage can propagate to downstream modules, leading to instability in optimization formulation, execution, and our benchmarking. To mitigate this issue, we employ a variant of MBR decoding that is discussed in Section 3.3.1.

3.3.1. HYBRID COMPONENT-WISE MBR AND LLM RE-RANKING

The goal of the decision process extractor is to produce a stable and reliable structured view of different components of a decision process, without requiring manual validation or access to ground-truth formulations. To mitigate the inherent non-determinism of reasoning LLMs, we adopt a parallel extraction strategy based on MBR decoding combined with lightweight LLM-based re-ranking. The core idea is to generate multiple candidate extractions in parallel and select a representative extraction that is maximally consistent with the others, thereby reducing inconsistencies and formatting variability.

Our hybrid MBR approach consists of two stages. In the first stage, we generate n candidate extractions, conditioned on the problem description D and retrieved memory context $\mathcal{M}^*$. Each candidate extraction $\mathcal{P}_i$ is represented as a collection of structured components $\{c_j^i\}_{j=1}^J$, where j indexes the component type. To quantify agreement across candidates, we compute component-wise utility scores based on semantic similarity. Each component is embedded using a dense embedding model, and similarity between components is measured via cosine similarity. For a given component type

j of the i -th candidate, its similarity to the corresponding components from other candidates is defined as

$$S(c_j^i) = \frac{1}{n-1} \sum_{\substack{k=1 \\ k \neq i}}^n \text{sim}(c_j^k, c_j^i).$$

The overall utility score for candidate $\mathcal{P}_i$ is computed as a weighted sum of its component utilities,

$$U(i) = \sum_{j=1}^J w_j S(c_j^i)$$

with fixed weights $w_j \geq 0$ such that $\sum_{j=1}^J w_j = 1$. These weights quantify the relative importance of the mathematical components of the formulation (e.g., constraints vs. variables). In our experiments, we fix the weights w_j across all candidates to reflect the relative contribution of each component type; details of the weight settings are provided in Appendix C.1.

Based on these utilities, we select the indices of the top- q extractions as

$$\mathcal{I}_{\text{top-}q} = \arg \max_{\substack{\mathcal{I} \subseteq \{1, \dots, n\} \\ |\mathcal{I}|=q}} \sum_{i \in \mathcal{I}} U(i).$$

In the second stage, from this subset of extractions, a final extraction is chosen using an LLM-based logical verifier that assesses mathematical consistency, constraint completeness, and overall formulation soundness,

$$\mathcal{P}^* = \text{LLM-Judge}(\{\mathcal{P}_i : i \in \mathcal{I}_{\text{top-}q}\}, D).$$

We intentionally restrict the LLM-Judge to the original problem description D , rather than the full memory context, to

avoid biasing the final selection toward any particular retrieved example and to ensure that the chosen extraction is logically consistent with the target problem specification. An overview of the complete pipeline is shown in Figure 4.

3.4. Solver Recommender

Given the extraction $\mathcal{P}^*$ and a set of available solvers and frameworks, $\mathcal{SO}$, the solver recommender leverages a reasoning model to generate a ranked list of suitable solvers for solving the problem (together with certain usage and installation guidelines) as $\mathcal{R} : (\mathcal{P}^*, \mathcal{SO}) \rightarrow \mathcal{SO}^*$, where $\mathcal{SO}^* = \{(s_1, r_1, p_1), (s_2, r_2, p_2), \dots, (s_m, r_m, p_m)\}$ with s_i representing a solver, r_i being the rank of the solver (lower the better) and p_i denoting its suitability reasoning and other accompanying information.

3.5. Simulator

Given a natural-language problem description D and the extracted decision process components $\mathcal{P}^*$, we construct an executable simulator that evaluates candidate decision variables against the implied process dynamics and constraints. To this end, we remotely provide instructions in a carefully constructed prompt to the ACA to generate the simulator as a self-contained Python package, defined as the mapping $\mathcal{G}_{\text{sim}} : (D, \mathcal{P}^*) \rightarrow \mathcal{S}$, where $\mathcal{S}$ denotes the resulting executable simulator.

The simulator is designed to mimic the practitioner’s internal mental model of the decision process. Given a candidate assignment to the decision variables, the coding agent orchestrates execution of $\mathcal{S}$ and returns a structured evaluation consisting of feasibility status, detected constraint violations, and the incurred objective value. This execution-based feedback provides a concrete, model-grounded signal that is used for downstream validation and refinement.

Formally, the simulator implements a mapping

$$\mathcal{S} : \mathbb{R}^{|X|} \rightarrow \{0, 1\} \times (\mathbb{R} \cup \{\infty\}),$$

$$\mathcal{S}(x) = (\text{feasible}(x), F_{\text{sim}}(x)),$$

where $x \in \mathbb{R}^{|X|}$ denotes an assignment to the decision variables and $F_{\text{sim}}(x)$ denotes the corresponding objective value computed by the simulator (set to ∞ if infeasible). When infeasibility is detected, the simulator reports the violated constraints and associated diagnostic information, which is subsequently used in the asymmetric validation loop described in Section 3.7.

3.6. Optimizer

Analogous to the simulator, the optimizer is generated and executed through an ACA as a self-contained Python package. Given the extracted decision process components $\mathcal{P}^*$, solver recommendations $\mathcal{SO}^*$ (provided in the prompt), and

retrieved code artifacts from $\mathcal{M}^*$ (uploaded to the ACA sandbox), the ACA constructs an executable optimizer via the mapping $\mathcal{G}_{\text{opt}} : (\mathcal{P}^*, \mathcal{SO}^*, \mathcal{M}^*) \rightarrow \mathcal{O}$, where $\mathcal{O}$ denotes the resulting optimization package.

Once generated, the ACA orchestrates the execution of $\mathcal{O}$ to solve the underlying optimization problem. This process includes invoking the selected solver, post-processing solver outputs, interpreting results, and collecting diagnostic information. The optimizer returns the optimal decision variables, solver termination status, and the corresponding objective value. To further improve the performance and robustness of the optimizer, we employ a self-consistency mechanism based on the computed decision variables, objective values, and solver status, described in Section 3.6.1.

3.6.1. SELF-CONSISTENCY FOR SOLUTION TRAJECTORIES

We construct T optimization implementations in parallel using the ACA. Each implementation produces a candidate solution $x_i \in \mathbb{R}^{|X|}$, along with associated solver metadata. We then select a robust solution through a hierarchical consensus procedure that aggregates solver status, objective value, and decision variables.

We first determine a consensus solver status using majority voting across the T runs. In the event of ties, we apply a lexicographic tie-breaking rule: *Optimal* $\succ$ *Time Limit* $\succ$ *Infeasible* $\succ$ *Unbounded* $\succ$ *Error*. This ordering favors potentially valid solutions—even if suboptimal due to time limits—over definitive failure modes.

If the consensus status is *Optimal* (or *Time Limit*), we further group solutions sharing this status based on their objective values, relying on numerical similarity. Two objective values $F_{\text{opt}}(x_i)$ and $F_{\text{opt}}(x_j)$ are considered similar if

$$|F_{\text{opt}}(x_i) - F_{\text{opt}}(x_j)| \leq \text{atol} + \text{rtol} \cdot |F_{\text{opt}}(x_j)|,$$

with $\text{rtol} = 10^{-6}$ and $\text{atol} = 10^{-9}$. The consensus objective value $F_{\text{opt}}(x^*)$ is selected as the median of the largest similarity group, reducing sensitivity to floating-point noise. The final decision vector x^* is taken from the implementation corresponding to this median; if multiple implementations achieve the median, we select the one with the lowest solver runtime to favor efficiency. This mechanism is executed automatically by the ACA and stabilizes optimizer performance.

3.7. Asymmetric Validation via Simulator–Optimizer Feedback

A key technical insight underlying NEMO is the complexity gap between verification (simulation) and solving (optimization). While constructing an optimizer requires translating natural language into complex declarative mathematical con-

straints, constructing a simulator typically involves writing imperative Python code that directly reflects the problem logic and is empirically less prone to translation errors (Section 4.4). To further ensure reliability, we instruct the ACA to generate not only the simulator $\mathcal{S}$ but also a comprehensive suite of unit tests (implemented via `pytest`), derived from both the problem description D and the extracted formulation $\mathcal{P}^*$. The simulator is used as a validation reference only if it passes these self-generated consistency checks.

NEMO leverages this validated simulator through an asymmetric cross-validation loop. Given a candidate solution x^* and objective value $F_{\text{opt}}(x^*)$ produced by the optimizer, the simulator provides an independent execution-based evaluation:

$$\mathcal{S}(x^*) = (\text{feasible}(x^*), F_{\text{sim}}(x^*)),$$

$$V(x^*) = \begin{cases} 1, & \text{if feasible}(x^*) = 1 \text{ and} \\ & |F_{\text{sim}}(x^*) - F_{\text{opt}}(x^*)| \leq \delta, \\ 0, & \text{otherwise,} \end{cases}$$

where $\delta = \text{atol} + \text{rtol} \cdot |F_{\text{opt}}(x^*)|$ is the numerical tolerance threshold. A validation outcome $V(x^*) = 1$ indicates consistency between the optimizer’s declarative formulation and the simulator’s validated imperative logic.

When validation fails ($V(x^*) = 0$), the simulator produces a structured error report describing violated constraints or objective mismatches. This report is injected into the optimizer ACA’s context as a refinement prompt, explicitly instructing the agent to debug the optimization model against the reported failures. The optimizer then generates a revised implementation, forming a self-correcting feedback loop driven by execution artifacts rather than manual intervention.

4. Experiments

4.1. Experimental Setup

We evaluate NEMO on nine established optimization benchmarks spanning diverse problem domains and complexity levels; specifications are provided in Appendix A.1. Throughout the paper, we distinguish between the Standard benchmarks (the original benchmark distributions released in prior work) and the Curated benchmarks (a curated version obtained via dataset curation and quality control; see Appendix A.2). To ensure fair comparison with prior baselines, all results reported in the main text (Table 1) are evaluated on the Standard Benchmarks unless otherwise noted.

All experiments utilize a unified system configuration: OpenAI’s o3 model serves as the primary reasoning LLM, while *OpenHands* (powered by Claude 3.7 Sonnet) acts as the ACA. Qwen3-Embedding-8B (Zhang et al., 2025b) drives

MBR and memory retrieval. To assess generalization, we avoid benchmark-specific hyperparameter tuning. Instead, system parameters (listed in Appendix C.1) were selected based on qualitative monitoring of a small set of development instances and applied uniformly across all tasks. This protocol emphasizes robustness and transferability over narrow optimization.

We compare NEMO against state-of-the-art agent-based frameworks, OptimAI, OptiMUS, OR-LLM-Agent, and Chain-of-Experts (CoE), and training-based methods such as ORLM, SIRL, OptMATH, and LLMOPT. For each baseline, we report the most recent publicly available results, taken from either the corresponding publication or the associated repository, whichever yields the strongest performance. Following prior work (Chen et al., 2025), we measure accuracy by comparing the objective value produced by the system, $F_{\text{opt}}(x^*)$, to the ground-truth optimal objective $F(x^{\text{gt}})$; see Appendix C.2 for full details.

4.2. Main Results

Table 1 summarizes performance across the nine benchmarks, comparing NEMO against state-of-the-art agent-based and training-based approaches.

To ensure rigorous and fair comparison, we report results on the Standard Benchmarks as well as on the specific curated test sets released by prior works (e.g., SIRL, LLMOPT) where applicable. Overall, NEMO achieves strong and consistent performance, ranking first or tied for first on eight of the nine benchmarks under at least one evaluation setting, and outperforming prior methods by large margins on several datasets. To facilitate transparency and reproducibility, we release granular intermediate outputs from the different components of NEMO via HuggingFace². We hope that this release will enable deeper analysis of system behavior and encourage further investigation and development of execution-aware approaches for language-driven optimization.

Remark. *We adopt the experimental protocols reported in the prior works we benchmark against. These approaches do not report statistical significance or variability measures (e.g., multiple runs or confidence intervals), and we follow the same practice for the headline results in Table 1, as the ACA execution loop is computationally intensive and repeated end-to-end evaluations across all nine benchmarks would be prohibitive. To nevertheless bound run-to-run variability, we conducted 5 independent evaluation runs on BWOR and ComplexOR—the two smallest benchmarks, and thus the most susceptible to stochastic variation—yielding standard deviations of $\pm 1.3\%$ and $\pm 2.5\%$, respectively. These values fall well within NEMO’s margins over prior*

²<https://huggingface.co/datasets/c3aia3c/nemo-icml2026>

Dataset	Standard Benchmarks									Curated (LLMOPT)		Curated (SIRL)	
	NEMO	OptimAI	OptiMUS	OR-LLM-Agent	CoE	OptMATH	ORLM	LLMOPT	SIRL	NEMO	LLMOPT	NEMO	SIRL
OptiBench	90.4%	82.3%	-	-	-	66.1%	-	66.4%	67.4%	-	-	-	-
OptMATH-Bench	65.7%	-	-	-	-	34.7%	-	40.0%	45.8%	-	-	-	-
NL4OPT	98.4%	-	78.8%	75.9%	64.2%	95.9%	86.5%	-	-	99.1%	97.3%	98.7%	98.4%
NLP4LP	81.4%	-	72.0%	-	53.1%	-	-	-	-	95.7%	86.5%	-	-
BWOR	82.9%	-	-	82.9%	-	-	-	-	-	-	-	-	-
IndustryOR	63.0%	-	-	36.0%	-	31.0%	38.0%	44.0%	-	-	-	76.0%	48.0%
MAMO-Easy	83.4%	-	-	82.2%	-	89.9%	85.2%	-	-	92.5%	95.3%	93.5%	94.7%
MAMO-Complex	72.0%	-	-	51.6%	-	54.1%	44.1%	85.8%	-	-	-	94.0%	72.4%
ComplexOR	77.8%	-	66.7%	-	38.1%	-	-	72.7%	-	-	-	-	-

Table 1. Benchmark results on 9 datasets. We report accuracy on Standard benchmarks and Curated variants released by prior work (LLMOPT, SIRL). NEMO achieves strong performance across both settings, consistently improving over prior agent-based baselines and remaining competitive with training-based methods without task-specific fine-tuning. Bold denotes the best result per dataset and setting¹.

Dataset	NEMO w/o Sim	NEMO (Base)	NEMO +Mem	NEMO +Mem+MBR	NEMO +Mem+MBR+Multi
OptMATH	59.6%	63.2%	63.9%	64.5%	65.7%
BWOR	71.9%	75.6%	80.4%	82.9%	82.9%
IndustryOR	60.0%	60.0%	62.0%	63.0%	63.0%
MAMO-Complex	50.9%	54.2%	61.9%	71.0%	72.0%
NLP4LP	76.0%	77.5%	79.0%	81.4%	81.4%

Table 2. Ablation study of system components across five benchmarks. Results are reported for progressively augmented variants, starting from a variant without the simulator (NEMO w/o Sim) and incrementally adding the simulator, memory, MBR decoding, and multiple optimizer backends. Bold denotes the best result per dataset.

SOTA (up to 20 percentage points), indicating that the reported gains reflect architectural contributions rather than stochastic noise, and that the MBR and self-consistency protocols effectively stabilize ACA non-determinism.

Across agent-based baselines, NEMO consistently improves upon prior coordination-based approaches. Notably, on OptiBench and OptMATH-Bench, our system achieves absolute accuracy gains of over 8 and approximately 20 percentage points, respectively, compared to the strongest reported baselines. On BWOR, our method matches the best prior result while maintaining high consistency across problem variants.

When compared to training-based methods, NEMO remains competitive or superior despite relying on general-purpose language models without domain-specific fine-tuning. Most notably, on the curated IndustryOR benchmark, NEMO outperforms SIRL by a remarkable margin of 28 percentage points, highlighting the substantial advantage of execution-aware validation and memory-based adaptation in handling complex, real-world optimization tasks.

4.3. Ablation Study

Table 2 presents an ablation study examining the contribution of key system components across five benchmarks spanning combinatorial optimization (BWOR), mathematical

²Note that OptimAI reported accuracy on NLP4LP using only the 65 LP problems, whereas we evaluate on the full dataset containing 269 LP and MILP problems.

programming (OptMATH-Bench, NLP4LP), multi-domain competition problems (MAMO-Complex), and industry-grade scenarios (IndustryOR)³.

Across all benchmarks, the core system components provide consistent, incremental improvements as they are progressively layered in. The simulator contributes positively on the four non-IndustryOR benchmarks (3.7 percentage points on BWOR, 3.6 on OptMATH-Bench, 3.3 on MAMO-Complex, and 1.5 on NLP4LP), and yields no net gain on IndustryOR.

To understand the IndustryOR exception, we compare NEMO Base and NEMO w/o Sim at the instance level on the 85 evaluable problems (excluding 15 with invalid ground truths). 84% of failures (21 of 25) are shared between the two variants, with root causes uniformly upstream of the simulator’s scope: extraction errors (52%), logic/constraint errors (43%), and solver failures (5%). The simulator does correct all 5 implementation-level bugs it encounters (5 of 5 detected, 4 of 5 resolved); the dominant upstream failures are instead addressed by MBR and Memory, which is why the full NEMO system still achieves 28 percentage points over prior SOTA on IndustryOR despite the simulator’s zero marginal contribution in the ablation. A complementary failure-mode breakdown for the complete NEMO pipeline is provided in Figure 6 of Appendix D; further sensitivity analyses are in Appendix B.

4.4. Complexity Asymmetry between Simulation and Optimization

To validate the architectural premise that simulation is a systematically simpler task than optimization, we evaluate single-shot Pass@1 of code generation for both tasks on 100 OptiBench problems against human-verified ground truths (not agent-generated unit tests). A pass is recorded when the generated code reproduces the ground-truth objective value within numerical tolerance δ ; the simulator additionally must satisfy all stated constraints upon execution. LLMs achieve 97% Pass@1 for procedural simulators versus 87%

³ComplexOR is excluded because NEMO (Base) already saturates to 100% accuracy on the curated dataset (see Appendix A.2), preventing meaningful component analysis due to ceiling effects.

Method	BWOR	OptMATH-Bench
Best-of-5 ACA Baseline	79.3%	62.7%
NEMO	86.6%	68.1%
Δ (percentage points)	+7.3	+5.4

Table 3. Compute-matched Best-of-5 ACA baseline vs. NEMO (Claude 4.5 Sonnet). Both conditions use the same backend and comparable token budgets. NEMO’s gains come from its execution-grounded architecture, not compute volume.

for declarative optimizers, a gap of 10 percentage points. This margin confirms simulation as a systematically simpler task and justifies its use as the high-confidence verification reference in the asymmetric validation loop (Section 3.7).

4.5. Compute-Matched Baseline and Full-Stack Simulator Contribution

To verify that NEMO’s gains stem from its execution-grounded architecture rather than increased inference-time compute, we construct a Best-of-5 ACA baseline using the same backend (Claude 4.5 Sonnet) with 5 independent single-pass ACA calls, selecting the best-objective solution. Despite matching the compute budget, NEMO outperforms this baseline by substantial margins (Table 3), confirming that structured execution-grounded feedback—not token volume—drives the improvements.

We additionally isolate the marginal contribution of the asymmetric validation loop within the fully assembled system. Disabling only the simulator from the complete NEMO pipeline (Memory + MBR + Multi-Optimizers) on Claude 4.5 Sonnet drops BWOR accuracy from 86.6% to 80.5% (a drop of 6.1 percentage points), confirming that the simulator continues to provide significant independent value even atop all other components.

5. Related Work

5.1. LLM-Based Optimization Modeling

Recent advances in LLMs have enabled significant progress toward automating optimization modeling from natural-language problem descriptions. Existing approaches broadly fall into two categories: agent-based frameworks and training-based methods.

Agent-based frameworks mitigate the complexity of optimization modeling by decomposing the workflow into specialized, coordinated sub-tasks. Chain-of-Experts (CoE) (Xiao et al., 2024) introduces a cooperative ecosystem where agents assume distinct reasoning roles, synchronized through iterative reflection. Similarly, pipelines like OptiMAI (Thind et al., 2025) and OR-LLM-Agent (Zhang et al., 2025a) structure the process into sequential stages, ranging from formulation to execution, often relying on coder-critic

interactions to refine outputs. Similarly, OptiMUS (Ahmaditeshnizi et al., 2024) prioritizes modularity to facilitate scalable Mixed-Integer Linear Programming (MILP) formulation. While these approaches successfully demonstrate the utility of specialization, they often necessitate intricate coordination protocols. Furthermore, their reliance on iterative critic agents can introduce fragility, as error propagation across stages remains a significant challenge when validation mechanisms are not grounded in execution.

Training-based approaches aim to internalize optimization knowledge directly into model parameters. ORLM (Huang et al., 2025a) performs supervised fine-tuning of open-source LLMs using synthetic instruction data generated via the OR-Instruct framework. Solver-Informed Reinforcement Learning (SIRL) (Chen et al., 2025) leverages external optimization solvers as verifiers, providing reward signals related to syntax validity, feasibility, and solution quality during training. LLMOPT (Jiang et al., 2025) combines multi-instruction supervised fine-tuning with model alignment techniques to generate structured optimization formulations and solver code. Similarly, OptMATH (Lu et al., 2025) introduces a comprehensive instruction-tuning dataset derived from semi-structured optimization problems, enabling models to better bridge the gap between natural-language descriptions and mathematical formulations. While training-based methods can reduce hallucinations and improve consistency and performance, they require substantial computational resources and typically exhibit limited transferability to new optimization domains without additional retraining.

5.2. Positioning of NEMO

NEMO’s Workspace-State paradigm (Section 2.1) directly addresses the core limitations of prior message-passing systems: agents share a persistent workspace rather than exchanging structured text, enabling native execution, cross-component imports, and executable memory without schema negotiation. Building on this, the system employs an asymmetric validation loop in which an independently generated simulator serves as an executable reference for validating optimizer outputs, enabling systematic detection of logical inconsistencies and implementation errors without ground-truth formulations. In contrast to training-based methods, this design avoids domain-specific fine-tuning and adapts to new optimization domains through memory expansion rather than retraining.

6. Limitations and Future Work

Formulation correctness and formal verification. Objective-value agreement with ground truth is a necessary but not sufficient condition for formulation correctness: two models may agree on the optimal objective while dif-

fering in their constraint sets or feasible regions. Formally verifying semantic equivalence between natural-language specifications and optimization models is an open problem in program synthesis; NEMO’s asymmetric validation loop provides a practical mitigation by catching structural implementation errors through cross-paradigm execution checks, but full formal verification remains future work.

Computational overhead and inference-time trade-offs.

A primary limitation of our ACA-based pipeline is computational cost. Compared to direct solver calls or single-pass LLM generation, our approach incurs additional overhead from iterative code generation, sandbox execution, and validation loops, taking 5–10 minutes per instance. Per-module latency profiling shows that approximately 55% of wall-clock time is irreducible ACA reasoning (inherent to any agent-based system), 32% is NEMO-specific overhead (validation loop and orchestration), and 13% is the cold-start phase (extraction, solver recommendation, memory retrieval). While this latency is acceptable when optimizer construction is infrequent and artifacts are reused, it may be prohibitive for high-throughput applications. However, the consistent performance improvements suggest this computation represents a form of inference-time scaling, where increased reasoning effort yields higher solution quality. Future work should explore acceleration strategies including caching code templates, parallelizing independent ACA runs, and distilling recurring patterns into specialized components.

Learning from execution-based validation. Existing reinforcement learning approaches for optimization modeling rely on comparing outputs against ground-truth solutions, providing coarse, outcome-level signals limited by data availability. In contrast, our simulator–optimizer validation loop yields richer, ground-truth-free execution-based feedback by cross-checking independently generated components. This signal exposes where and how errors arise through feasibility checks, objective consistency, and structured discrepancies, rather than merely whether outputs match known answers. Leveraging this execution-grounded feedback as a learning signal represents a promising direction for improving the robustness and scalability of language-driven optimization systems.

7. Conclusion

We introduced NEMO, an execution-aware system for translating natural-language descriptions of decision problems into executable mathematical optimization programs using ACAs. In contrast to prior approaches based on direct LLM code generation or bespoke critic pipelines, NEMO treats ACAs as first-class primitives and leverages asymmetric validation between independently generated simulators and

optimizers to systematically detect and correct modeling errors, augmented by memory-based few-shot learning, MBR decoding, and self-consistency mechanisms.

Across nine optimization benchmarks, NEMO achieves strong performance, ranking first or tied for first on eight benchmarks under at least one evaluation setting, with substantial improvements on complex real-world problems. These gains are achieved without domain-specific training, benchmark-specific tuning, or reliance on frontier models, demonstrating the effectiveness of execution-aware validation over training-intensive alternatives.

Beyond modeling, this work introduces a novel interaction paradigm with ACAs through execution-grounded workflows, integrating generation, validation, and iterative refinement, offering a robust architectural template for agentic systems in high-stakes domains requiring correctness and coordinated reasoning.

Impact Statement

This paper presents work aimed at democratizing access to optimization modeling by lowering technical barriers for practitioners without specialized operations research training. While this could enable more efficient resource allocation across healthcare, logistics, and public services, we acknowledge several considerations. The system is designed as a decision-support tool requiring human oversight, particularly in high-stakes domains, as automated approaches can produce incorrect solutions for problems outside their training distribution. The computational overhead of our inference-time scaling approach (5-10 minutes per instance) may result in substantial energy consumption at scale. Additionally, while optimization technologies are fundamentally neutral, automated modeling could be applied to contexts raising ethical concerns, such as surveillance or algorithmic decision-making systems. We have also documented significant data quality issues in existing benchmarks (87.3% retention rate), highlighting the need for high-quality, diverse evaluation datasets to ensure robust performance. Users should understand system limitations and maintain appropriate human validation, especially for critical applications.

References

Ahmaditeshnizi, A., Gao, W., and Udell, M. OptiMUS: Scalable optimization modeling with (MI)LP solvers and large language models. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pp. 577–596. PMLR, 2024. URL <https://proceedings.mlr.press/v235/ahmaditeshnizi24a.html>.

Antonioni, A. and Lu, W.-S. *Practical Optimization: Algo-*

- rithms and Engineering Applications*. Texts in Computer Science. Springer, New York, NY, 2 edition, 2021. ISBN 978-1-0716-0843-2.
- Brown, T. B., Mann, B., Ryder, N., Subbiah, M., Kaplan, J., Dhariwal, P., Neelakantan, A., Shyam, P., Sastry, G., Askell, A., Agarwal, S., Herbert-Voss, A., Krueger, G., Henighan, T., Child, R., Ramesh, A., Ziegler, D. M., Wu, J., Winter, C., Hesse, C., Chen, M., Sigler, E., Litwin, M., Gray, S., Chess, B., Clark, J., Berner, C., McCandlish, S., Radford, A., Sutskever, I., and Amodei, D. Language models are few-shot learners. In *Advances in Neural Information Processing Systems*, volume 33, pp. 1877–1901, 2020.
- Chen, Y., Xia, J., Shao, S., Ge, D., and Ye, Y. Solver-informed RL: Grounding large language models for authentic optimization modeling. In *Advances in Neural Information Processing Systems*, 2025. URL <https://neurips.cc/virtual/2025/poster/119660>.
- Cornuéjols, G., Peña, J., and Tütüncü, R. *Optimization Methods in Finance*. Cambridge University Press, Cambridge, 2 edition, 2018. ISBN 9781107056749.
- DeCroix, G. A., Long, X., and Tong, J. How service quality variability hurts revenue when customers learn: Implications for dynamic personalized pricing. *Operations Research*, 69(3):683–708, 2021. doi: 10.1287/opre.2020.2058.
- Huang, C., Tang, Z., Hu, S., Jiang, R., Zheng, X., Ge, D., Wang, B., and Wang, Z. ORLM: A customizable framework in training large models for automated optimization modeling. *Operations Research*, 73(6):2986–3009, 2025a.
- Huang, X., Shen, Q., Hu, Y., Gao, A., and Wang, B. LLMs for mathematical modeling: Towards bridging the gap between natural and mathematical languages. In *Findings of the Association for Computational Linguistics: NAACL 2025*, pp. 2678–2710, Albuquerque, New Mexico, April 2025b. Association for Computational Linguistics. doi: 10.18653/v1/2025.findings-naacl.146. URL <https://aclanthology.org/2025.findings-naacl.146/>.
- Jiang, C., Shu, X., Qian, H., Lu, X., Zhou, J., Zhou, A., and Yu, Y. LLMOPT: Learning to define and solve general optimization problems from scratch. In *Proceedings of the Thirteenth International Conference on Learning Representations (ICLR)*, Singapore, Singapore, 2025. URL <https://openreview.net/pdf?id=90MvtboTJg>.
- Li, X. L. and Liang, P. Prefix-tuning: Optimizing continuous prompts for generation. In *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (Volume 1: Long Papers)*, pp. 4582–4597. Association for Computational Linguistics, 2021. doi: 10.18653/v1/2021.acl-long.353. URL <https://aclanthology.org/2021.acl-long.353/>.
- Liu, P., Yuan, W., Fu, J., Jiang, Z., Hayashi, H., and Neubig, G. Pre-train, prompt, and predict: A systematic survey of prompting methods in natural language processing. *ACM Comput. Surv.*, 55(9), January 2023. ISSN 0360-0300. doi: 10.1145/3560815. URL <https://doi.org/10.1145/3560815>.
- Lu, H., Xie, Z., Wu, Y., Ren, C., Chen, Y., and Wen, Z. OptMATH: A scalable bidirectional data synthesis framework for optimization modeling. In *Forty-second International Conference on Machine Learning*, 2025. URL <https://openreview.net/forum?id=9P5e6iE4WK>.
- OpenAI. GPT-4 technical report, 2023. URL <https://arxiv.org/abs/2303.08774>.
- Powell, W. B. *Reinforcement Learning and Stochastic Optimization: A Unified Framework for Sequential Decisions*. John Wiley & Sons, Hoboken, NJ, 2022. ISBN 9781119815037.
- Ramamonjison, R., Yu, T., Li, R., Li, H., Carenini, G., Ghaddar, B., He, S., Mostajabdaveh, M., Banitalebi-Dehkordi, A., Zhou, Z., and Zhang, Y. NL4Opt competition: Formulating optimization problems based on their natural language descriptions. In Ciccone, M., Stolovitzky, G., and Albrecht, J. (eds.), *Proceedings of the NeurIPS 2022 Competitions Track*, volume 220 of *Proceedings of Machine Learning Research*, pp. 189–203. PMLR, 28 Nov–09 Dec 2022. URL <https://proceedings.mlr.press/v220/ramamonjison23a.html>.
- Saghafian, S., Austin, G., and Traub, S. J. Operations research/management contributions to emergency department patient flow optimization: Review and research prospects. *IIE Transactions on Healthcare Systems Engineering*, 5(2):101–123, 2015. doi: 10.1080/19488300.2015.1017676. URL <https://doi.org/10.1080/19488300.2015.1017676>.
- Schick, T. and Schütze, H. Exploiting cloze-questions for few-shot text classification and natural language inference. In *Proceedings of the 16th Conference of the European Chapter of the Association for Computational Linguistics: Main Volume*, pp. 255–269. Association for Computational Linguistics, 2021. URL <https://aclanthology.org/2021.eacl-main.20/>.

- Shakoor, R., Hassan, M. Y., Raheem, A., and Wu, Y.-K. Wake effect modeling: A review of wind farm layout optimization using Jensen’s model. *Renewable and Sustainable Energy Reviews*, 58:1048–1059, 2016. ISSN 1364-0321. doi: 10.1016/j.rser.2015.12.229.
- Singh, A. An overview of the optimization modelling applications. *Journal of Hydrology*, 466-467:167–182, 2012. ISSN 0022-1694. doi: 10.1016/j.jhydrol.2012.08.004.
- Thind, R., Sun, Y., Liang, L., and Yang, H. Optimai: Optimization from natural language using llm-powered ai agents, 2025. URL <https://arxiv.org/abs/2504.16918>.
- Wang, X., Li, B., Song, Y., Xu, F. F., Tang, X., Zhuge, M., Pan, J., Song, Y., Li, B., Singh, J., Tran, H. H., Li, F., Ma, R., Zheng, M., Qian, B., Shao, Y., Muennighoff, N., Zhang, Y., Hui, B., Lin, J., Brennan, R., Peng, H., Ji, H., and Neubig, G. OpenHands: An open platform for AI software developers as generalist agents. In *International Conference on Learning Representations*, 2025.
- Xiao, Z., Zhang, D., Wu, Y., Xu, L., Wang, Y. J., Han, X., Fu, X., Zhong, T., Zeng, J., Song, M., and Chen, G. Chain-of-Experts: When LLMs meet complex operations research problems. In *The Twelfth International Conference on Learning Representations*, 2024. URL <https://openreview.net/forum?id=HobyL1B9CZ>.
- Yang, Z., Wang, Y., Huang, Y., Guo, Z., Shi, W., Han, X., Feng, L., Song, L., Liang, X., and Tang, J. OptiBench meets ReSocratic: Measure and improve LLMs for optimization modeling. In *The Thirteenth International Conference on Learning Representations*, 2025. URL <https://openreview.net/forum?id=fsDZws49uY>.
- Zhang, B., Luo, P., Yang, G., Soong, B.-H., and Yuen, C. OR-LLM-Agent: Automating modeling and solving of operations research optimization problems with reasoning LLM, 2025a. URL <https://arxiv.org/abs/2503.10009>.
- Zhang, Y., Li, M., Long, D., Zhang, X., Lin, H., Yang, B., Xie, P., Yang, A., Liu, D., Lin, J., Huang, F., and Zhou, J. Qwen3 embedding: Advancing text embedding and reranking through foundation models. *arXiv preprint arXiv:2506.05176*, 2025b.

A. Dataset

A.1. Dataset Description

We evaluate our system across nine operations research benchmark datasets. Dataset statistics are summarized in Table 4.

Dataset Name	# Questions	Problem Types
OptiBench (Yang et al., 2025)	605	LP, MILP, NLP
OptMATH-Bench (Lu et al., 2025)	166	LP, MILP, NLP, SOCP
NL4OPT (Ramamonjison et al., 2022)	245	LP
NLP4LP (Ahmaditeshnizi et al., 2024)	269	LP, MILP
BWOR (Zhang et al., 2025a)	82	LP, MILP
IndustryOR (Huang et al., 2025a)	100	LP, MILP
MAMO-Easy (Huang et al., 2025b)	652	LP, MILP
MAMO-Complex (Huang et al., 2025b)	211	LP, MILP, NLP
ComplexOR (Xiao et al., 2024)	18	LP, MILP

Table 4. Comparison of optimization benchmark datasets by size and problem type.

OptiBench: A collection of 605 optimization word problems sourced from university textbooks and open-source solver repositories. It spans a diverse range of mathematical formulations, including Linear Programming (LP), Mixed-Integer Linear Programming (MILP), and Non-Linear Programming (NLP).

OptMATH-Bench: Contains 166 challenging semi-structured instances derived from advanced mathematics competitions. The dataset is characterized by extended natural-language contexts and complex constraints covering Linear Programming (LP), Mixed-Integer Linear Programming (MILP), Non-Linear Programming (NLP), and Second-Order Cone Programming (SOCP).

NL4OPT: Comprises 245 Linear Programming (LP) problems synthetically generated for the NeurIPS 2022 competition. It focuses on the precise translation of natural-language descriptions into canonical linear constraints and objective functions.

NLP4LP: A dataset of 269 problems featuring long, intricate descriptions adapted from standard optimization libraries. While primarily focused on Linear Programming (LP), harder subsets include Mixed-Integer Linear Programming (MILP) instances that test extraction from dense technical specifications.

BWOR: Consists of 82 business-oriented problems sourced from classic Operations Research textbooks. These problems represent standard reasoning tasks involving Linear Programming (LP) and Mixed-Integer Linear Programming (MILP) formulations applied to typical business scenarios.

IndustryOR: An industrial benchmark containing 100 problems derived from real-world case studies in manufacturing, supply chain logistics, and finance. It focuses on practical applications requiring models to handle constraints common in industrial Linear Programming (LP) and Mixed-Integer Linear Programming (MILP) settings.

MAMO-Easy: A subset of the MAMO benchmark containing 652 problems collected from mathematical modeling competitions. These instances focus on fundamental algebraic and Linear Programming (LP) tasks suitable for evaluating basic solver capabilities.

MAMO-Complex: The difficult subset of the MAMO benchmark (211 problems), also sourced from modeling competitions. These instances involve intricate dependencies and often require advanced Linear, Mixed-Integer, or Non-Linear Programming (LP/MILP/NLP) formulations and multi-step reasoning.

ComplexOR: A small but highly challenging set of 18 problems involving complex Linear Programming (LP) and Mixed-Integer Linear Programming (MILP) scenarios. These expert-crafted instances are designed to stress advanced reasoning under intricate constraint dependencies.

A.2. Dataset Curation and Quality Control

A.2.1. CURATION METHODOLOGY

We applied a systematic three-stage pipeline: (1) automated validation to detect malformed problems, (2) manual inspection of edge cases, and (3) exclusion based on rigorous predefined criteria. To ensure consistency, all excluded problems were

independently reviewed by at least two domain experts.

A.2.2. EXCLUSION CRITERIA

Problems were excluded if they exhibited the following issues:

- **Malformed Problem Statements:** Descriptions that were incomplete or ambiguous, particularly those lacking necessary constraints, clear objective functions, or defined decision variables.
- **Invalid Reference Solutions:** Ground-truth solutions that were mathematically infeasible, violated explicit constraints (e.g., exceeding budget or capacity limits), or contained numerical anomalies (e.g., arbitrarily large constants like 10^{35} used as proxies for infinity).

A.2.3. DATASET STATISTICS

Dataset	Original	Excluded	Final	Retention
OptiBench	605	54	551	91.1%
OptMATH-Bench	166	44	122	73.5%
NL4OPT	245	3	242	98.8%
NLP4LP	269	22	247	91.8%
BWOR	82	10	72	87.8%
IndustryOR	100	15	85	85.0%
MAMO-Easy	652	89	563	86.3%
MAMO-Complex	211	58	153	72.5%
ComplexOR	18	4	14	77.8%
Total	2,348	299	2,049	87.3%

Table 5. Curation results across all benchmark datasets. Overall retention rate: 87.3%.

Exclusion Category	Count	Percentage
Malformed problem statements	83	27.8%
Ground-truth issues	216	72.2%
Total	299	100%

Table 6. Distribution of exclusion reasons across all datasets.

A.2.4. CURATION SENSITIVITY

Curation was strictly *error-blind*: instances were excluded solely on the grounds of mathematical invalidity (infeasible ground truths, malformed problem statements), with no reference to system outputs or per-method performance. The two exclusion categories—malformed problem statements (27.8%) and ground-truth issues (72.2%, Table 6)—are identifiable independently of any solver or model output.

To verify that headline results do not depend on curation, Table 1 (main body) reports all primary comparisons on the Standard (uncurated) benchmarks. NEMO ranks first or tied for first on eight of nine benchmarks in this setting, confirming that the gains are not an artifact of the curation process. The curated benchmarks (Tables 5–7) are provided for transparency and to enable fair comparison with prior works that evaluated on their own curated subsets; they consistently show the same relative ordering and larger absolute margins (as invalid ground truths are removed), but are not required for any of the paper’s primary claims.

A.2.5. REPRESENTATIVE EXCLUSION EXAMPLES

Below are two representative examples of rejected instances that were filtered out during this process.

Malformed Statement (MAMO-Easy)

PROBLEM: A manufacturing company produces two types of products, X and Y. The production cost for each unit of product X is \$5000, and for product Y, it's \$3000. The total units produced for both products combined cannot exceed 1000 due to capacity constraints. To meet the market demand, the combined units produced, calculated as 3 times the units of product X plus 2 times the units of product Y, must be at least 2000. Additionally, the difference between the units produced for product X and Y should be at least 150 to maintain a balanced portfolio. Given that all variables are integers due to the indivisible nature of physical goods, what is the minimum total production cost in dollars (rounded to nearest dollar) required to fulfill these requirements?

PROVIDED GROUND-TRUTH (GT): 3,230,000

ISSUE: The ground truth of 3,230,000 is a result because it indicates that the solver interpreted the Balance Constraint ($|x - y|$ being greater or equal to 150) as the more restrictive $x - y$ being greater than 150 (Case 1), rather than allowing for more of the cheaper product (Case 2), which results in 3,000,000 as the solution.

Unbounded Ground-Truth (IndustryOR)

PROBLEM: A company must allocate a \$10,000 weekly budget across three advertising platforms to maximize viewer reach: Z-tube (\$1,000/ad, 400,000 viewers), Soorchle (\$200/ad, 5,000 viewers), and Wassa (\$100/ad, 3,000 viewers). Constraints require: (1) at most 15 ads on Soorchle, (2) at most one-third of total ads on Wassa, and (3) at least 5% of total ads on Z-tube.

PROVIDED GROUND-TRUTH (GT):

```
{
  "variables": {
    "NumberAdsZTube": -0.0,
    "NumberAdsSoorchle": -0.0,
    "NumberAdsWassa": -0.0,
    "xZ": 1e+30,
    "xS": 1e+30,
    "xW": 1e+30
  },
  "objectives": 4.08e+35
}
```

ISSUE: The ground truth solution is unbounded whereas there exists a feasible solution to this problem.

B. Ablation Studies

B.1. Performance on Curated Benchmarks

Dataset	NEMO -Sim	NEMO (Base)	NEMO +Mem	NEMO +Mem+MBR	NEMO +Mem+MBR+Multi
OptMATH-Bench	81.1%	86.0%	86.8%	87.7%	89.3%
BWOR	81.9%	86.1%	91.7%	94.3%	94.3%
IndustryOR	71.4%	71.4%	73.8%	75.0%	75.0%

Table 7. Ablation study showing performance across NEMO variants on the Curated benchmarks. Bold indicates best performance.

Table 7 details how each system component contributes to performance on the curated datasets. We utilize these verified

Dataset	Total Problems	Multi-Attempt Triggered	Resolved Correctly	Success Rate (Multi-Attempt)
OptMATH-Bench	166	8	5	62.5%
BWOR	82	2	2	100%
IndustryOR	100	5	4	80%

Table 8. Simulator-optimizer feedback loop activation and effectiveness across benchmarks. **Multi-Attempt Triggered** indicates problems requiring more than one iteration through the feedback loop. **Resolved Correctly** shows how many of these were ultimately solved. **Success Rate** measures the percentage of multi-attempt problems that were resolved correctly, demonstrating the effectiveness of the iterative refinement process.

benchmarks to measure the model’s true reasoning capabilities, isolating them from the noise caused by malformed or incorrect problems present in the original distributions.

The results demonstrate a clear, consistent trajectory of improvement as components are layered in. For instance, on the BWOR benchmark, the addition of Memory and MBR decoding steadily raises accuracy from 86.1% (Base) to 94.3%. Overall, these findings confirm that the system’s design elements function synergistically to enhance robustness. Furthermore, the significantly higher absolute scores compared to the standard benchmarks suggest that the performance gaps observed in Table 1 are largely attributable to data quality issues in the original sources rather than intrinsic limitations of the model.

B.2. Simulator-Optimizer Feedback Loop Analysis

Table 8 quantifies the simulator-optimizer feedback loop activation across three benchmarks. The feedback mechanism was triggered in only 1.2-5% of problems, indicating high initial solution quality. However, when activated, it achieved strong correction rates of 62.5-100%, successfully resolving 11 out of 15 initially incorrect solutions. This validates the simulator’s effectiveness both in detecting errors and guiding iterative refinement toward correct solutions.

B.3. Base Model in *OpenHands* (Claude 4.5 vs. Claude 3.7)

To understand how much our system benefits from stronger underlying models, we compared performance using Claude 3.7 Sonnet versus the more capable Claude 4.5 Sonnet. Table 9 details these results on both Standard and Curated benchmarks.

We observe that upgrading the base model yields consistent gains across all datasets. For example, on the Curated BWOR benchmark, accuracy rises to a near-perfect 98.6%. Importantly, these improvements occur in both the Standard and Curated settings. This confirms that NEMO scales effectively with better base models and that the performance boost from our agentic framework is complementary to advancements in the underlying LLM.

Dataset	Standard Benchmarks		Curated Benchmarks	
	Sonnet 3.7	Sonnet 4.5	Sonnet 3.7	Sonnet 4.5
OptMATH-Bench	65.7%	68.1%	89.3%	92.6%
BWOR	82.9%	86.6%	94.3%	98.6%
IndustryOR	63.0%	65.0%	75.0%	76.4%

Table 9. Impact of base model selection. We benchmark performance on both Standard and Curated datasets. Across both data regimes, upgrading the base model from Claude 3.7 to Claude 4.5 yields significant performance gains. Bold indicates best performance.

B.4. Vectorstore Analysis

Our memory bank contains 3,000 optimization problems spanning 15 distinct categories. Table 10 provides the taxonomy and definitions for these problem types.

To investigate potential data leakage and the robustness of our retrieval mechanism, we populated the vectorstore with 3,000 training samples from the OptMATH-train dataset. For each evaluation problem, we retrieved the top-5 relevant samples and analyzed the resulting similarity score distributions across all benchmarks. As detailed in Figure 3, the absence of similarity scores approaching 1.0 confirms that while the retriever identifies semantically relevant structures, it does not encounter exact duplicates or leaked test data, thereby ensuring the integrity of the evaluation.

Problem Type	Description
Knapsack	Select items to maximize value under capacity constraints
Assignment	One-to-one assignment of tasks, resources, or entities
Scheduling	Arrange timing and sequence of activities, tasks, or jobs
Transportation	Optimize shipment from sources to destinations
Facility Location	Decide where to open facilities to serve customers
Network Flow	Optimize resource flow in networks
TSP	Find shortest path visiting all nodes once
Vehicle Routing	Optimize delivery routes for multiple vehicles
Resource Allocation	Allocate limited resources among activities
Production Planning	Optimize production quantities and inventory
Inventory Management	Optimize inventory levels and ordering
Cutting Stock	Minimize material waste in cutting
Bin Packing	Pack items into minimum containers
Linear Programming	General linear optimization
Miscellaneous	Hybrid or uncategorized problems

Table 10. Taxonomy of problem types stored within the memory bank.

B.4.1. IMPACT OF DIVERSITY PARAMETER λ

Table 11 evaluates the impact of the diversity penalty λ in our memory retrieval scoring function. We compare a pure relevance-based strategy ($\lambda = 0.0$) against our diversity-aware approach ($\lambda = 0.5$). The results indicate that encouraging diversity is important for retrieving effective few-shot examples. On the BWOR dataset, introducing diversity improves accuracy by 13.4 percentage points, while IndustryOR exhibits an 11 percentage point gain. These findings suggest that simply retrieving the most similar examples often leads to redundant context, whereas enforcing $\lambda > 0$ promotes a broader and more representative set of problem-solving patterns, improving generalization.

Dataset	Diversity Parameter λ	
	$\lambda = 0.0$	$\lambda = 0.5$
OptMATH-Bench	60.0%	64.4%
BWOR	69.5%	82.9%
IndustryOR	52.0%	63.0%

Table 11. Ablation study on diversity parameter λ in the memory retrieval scoring function. $\lambda = 0$ prioritizes pure relevance while $\lambda > 0$ introduces diversity penalty. Bold indicates best performance for each dataset.

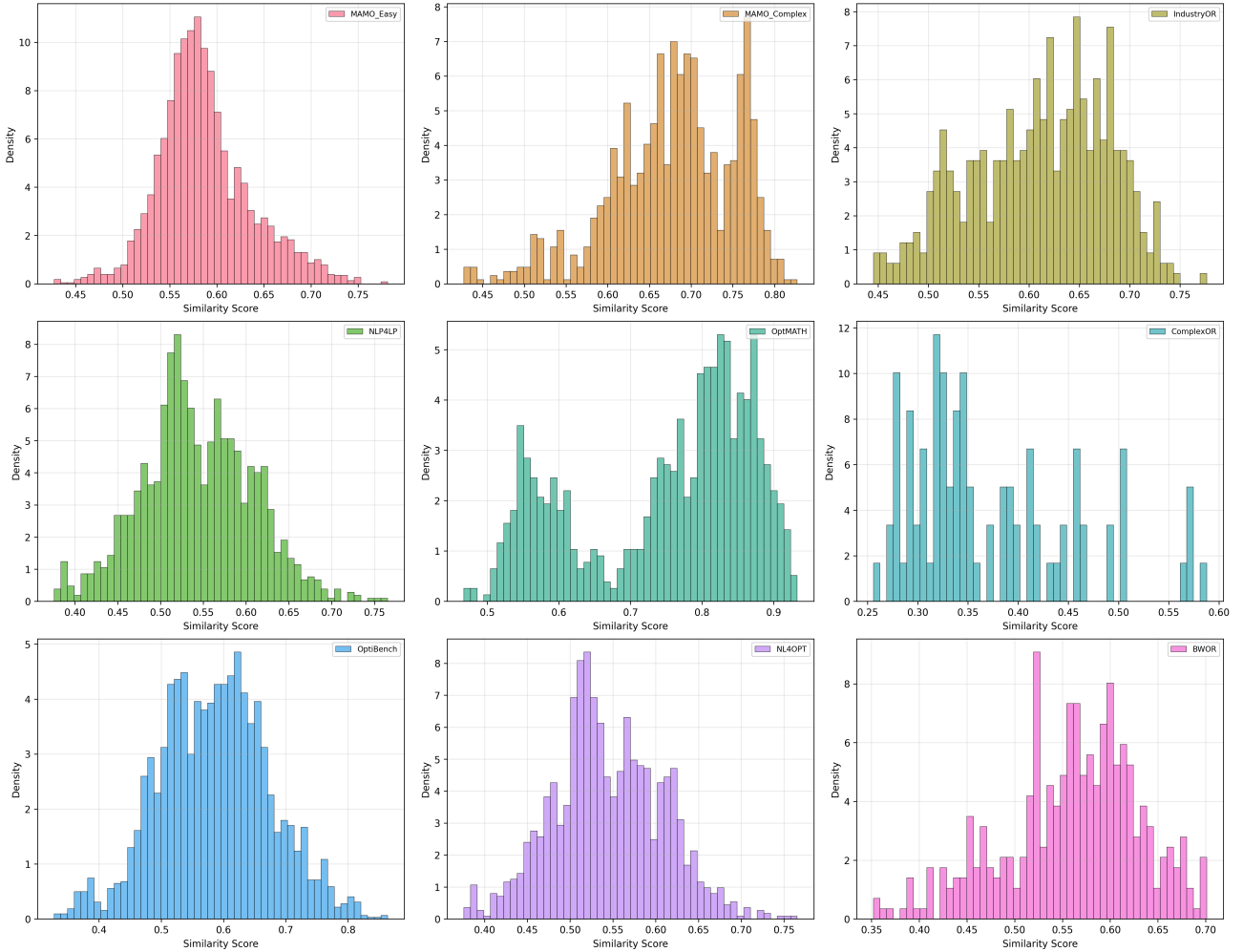


Figure 3. Distribution of top-5 similarity scores for nine evaluation benchmarks against the OptMATH training set. The distributions indicate a healthy semantic gap between the test queries and the stored training samples. The absence of high-density peaks near 1.0 confirms that no significant data leakage occurs, even when retrieving for the domain-adjacent OptMATH-Bench (center panel).

B.5. Consistency of MBR-Based Re-ranking

To reduce the stochasticity of LLM outputs, we employ MBR decoding. Figure 4 illustrates the hybrid pipeline: a fast embedding-based filter first removes weak candidates, allowing a stronger reasoning model to focus on a small set of promising solutions. We evaluate this approach using the scatter plot in Figure 5 and the quantitative results in Table 12.

The scatter plot reveals two distinct behaviors. First, most points cluster in the top-right quadrant, indicating that when the model is already confident, MBR agrees with standard sampling and preserves high-quality solutions. Second, a notable set of points appears in the top-left quadrant, corresponding to cases where vanilla sampling produces inconsistent outputs, but MBR successfully identifies a consensus solution.

Table 12 quantifies this stabilizing effect. By systematically filtering inconsistent outliers, MBR substantially reduces variance across extractions. For example, on OptMATH-Bench, stability improves by approximately a factor of three, indicating that MBR effectively mitigates random failures that arise under standard sampling.

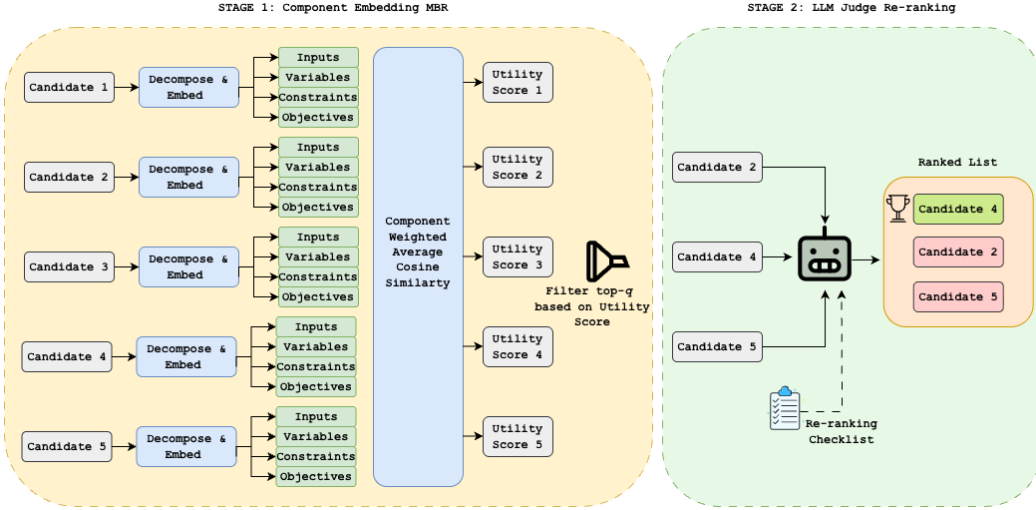


Figure 4. Hybrid component-wise MBR and LLM re-ranking pipeline. A fast embedding-based filter removes weak candidates early, allowing stronger reasoning models to be reserved for final top- q re-ranking, where semantic similarity is replaced by logical verification to select mathematically consistent extractions.

B.6. Solver Recommender Ablation

To isolate the contribution of the solver recommender, we run a Gurobi-only ablation on the 133 nonlinear-notable problems in OptiBench—the subset where solver selection most critically affects outcomes. Replacing the recommender with a fixed Gurobi backend drops accuracy from 84.2% to 76.7% (a drop of 7.5 percentage points), confirming that matching solvers to problem structure, particularly for non-linear constraints, is a significant driver of NEMO’s performance on this subset.

C. Experiment Configuration

C.1. Hyperparameter Configuration

Table 13 summarizes the global hyperparameter configuration used across all experiments. To ensure a consistent and reproducible evaluation, we fix a single set of parameters across all the benchmark datasets.

C.2. Evaluation Criteria

We evaluate solution accuracy by comparing the generated objective value $F_{\text{opt}}(x^*)$ against the ground-truth optimal objective $F(x^{\text{gt}})$. Following prior work (Chen et al., 2025), a solution is classified as correct if it satisfies the relative error criterion

$$\frac{|F_{\text{opt}}(x^*) - F(x^{\text{gt}})|}{|F(x^{\text{gt}})| + \epsilon} < 10^{-6},$$

where x^{gt} denotes the ground-truth optimal solution and ϵ is a small numerical stability constant, e.g., $\epsilon = 10^{-8}$.

In addition to satisfying this numerical threshold, we classify solutions as correct under the following well-defined exceptional cases, which arise from common ambiguities in benchmark formulations:

1. **Relaxation Mismatch.** The natural-language problem description implies discrete decision variables (e.g., counts of physical items), while the benchmark ground-truth is derived from a continuous LP relaxation. In such cases, solutions consistent with the relaxed formulation are considered correct.
2. **Verified Infeasibility.** The benchmark ground-truth indicates that the problem is infeasible, and the proposed solution independently proves infeasibility through execution-based validation.
3. **Equivalent Formulations.** The generated decision vector x^* is equivalent to x^{gt} , but the reported objective values differ due to alternative scaling or units of measurement (e.g., total cost reported in USD versus thousands of USD).

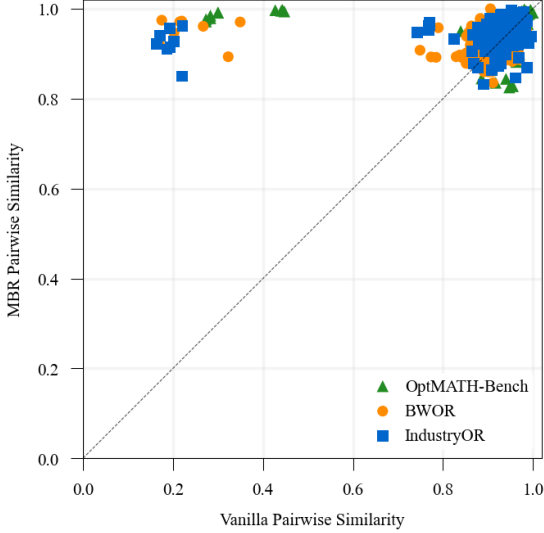


Figure 5. Scatter plot of raw pairwise similarity scores comparing vanilla sampling and MBR decoding. The dashed line indicates parity.

Dataset	Consistency $\uparrow$		Stability $\downarrow$	
	Vanilla	MBR	Vanilla	MBR
OptMATH-Bench	0.915	0.960	0.048	0.015
BWOR	0.895	0.935	0.057	0.023
IndustryOR	0.899	0.944	0.059	0.019

Table 12. Extraction variability analysis. *Consistency* denotes mean pairwise similarity ($\uparrow$), while *Stability* denotes intra-sample standard deviation ($\downarrow$).

D. Failure Modes Analysis

Through a granular analysis of 100 benchmark problems from IndustryOR, we identify that NEMO achieves a 63% success rate in generating valid and correct models. However, the remaining 37% of cases reveal critical failure modes categorized into modeling logic, external benchmark inconsistencies, and feasibility constraints. As shown in Figure 6, the primary bottleneck is **Wrong/Missing Constraints**, which accounts for 50% of all modeling-related errors. This indicates that while the system often identifies the correct objective, it may overlook the physical or logical boundaries inherent in complex industrial scenarios. Additionally, 12% of failures are attributed to **Upstream Inconsistencies** (Malformed Problem Statements or Incorrect Ground Truths), where the model’s output is penalized by artifacts within the benchmark data itself rather than logical derivation errors.

Category	Hyperparameter	Description	Value
Models	Reasoning LLM	Primary reasoning engine for NEMO	OpenAI o3
	ACA Backend	Base model for the OpenHands agent	Claude 3.7 Sonnet
	Embedding Model	Model for MBR and memory retrieval	Qwen3-Embedding-8B
	Batch Size	Number of instances per batch	5
Retrieval	Similarity Threshold	Minimum cosine similarity score	0.6
	Memory Pool Size ($ \mathcal{M} $)	Number of candidates initially retrieved	9
	Top- k Retrieved (k)	Number of examples selected for context	3
	Diversity (λ)	Balance between relevance and diversity	0.5
Extractor	Candidate Pool (n)	Total candidates generated	5
	Top- q Extractions	Candidates forwarded to LLM judge re-ranker	3
	Constraint Weight	Importance weight for constraints component	0.6
	Decision Variable Weight	Importance weight for decision variables	0.2
	Objective Weight	Importance weight for objective function	0.1
	Input Weight	Importance weight for input parameters	0.1
Optimizer	Optimizer Implementations (T)	Number of code implementations generated	3
	Maximum Validation Loops	Maximum optimizer validation iterations	3

Table 13. Hyperparameter configuration. The **Category** column groups settings by module. MBR component weights (**Extractor**) are normalized to sum to 1.0.

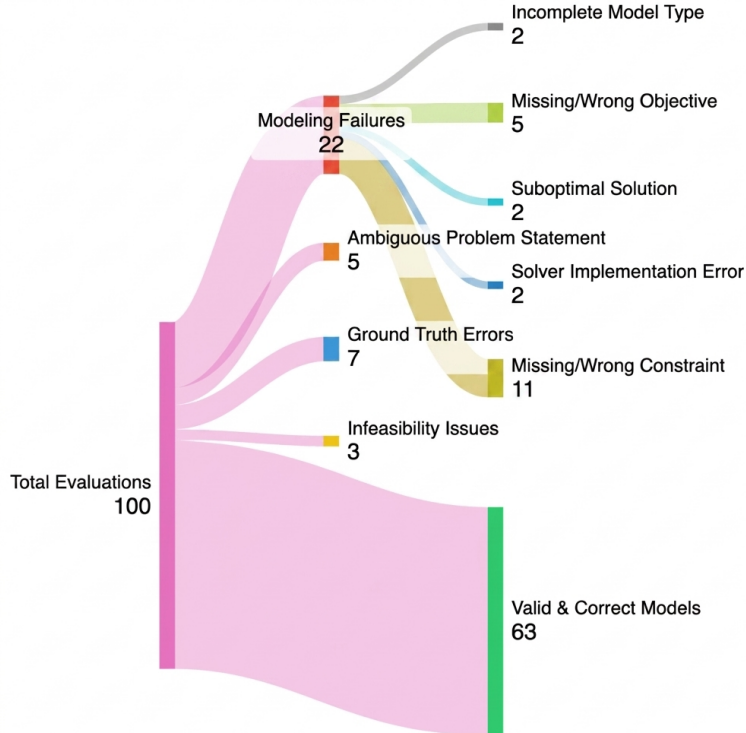


Figure 6. The distribution of failure modes in NEMO's optimization pipeline on the IndustryOR benchmark. The flow transitions from the total problem set into valid models and four primary categories of failure.

E. Module-Specific Prompts & End-to-End Execution Examples

In this section, we provide the system prompts used in our framework. While we shortened some text for brevity, all critical logic and rules are included. We also provide examples of the results generated by the system for a MAMO-Complex minimum cost network flow problem.

E.1. Decision Process Extractor

E.1.1. PROMPTS

Decision Process Extractor Prompt

Extract a complete mathematical specification of an optimization problem from a natural-language description.

Core logic:

- Interpret the problem strictly from `problem_description`.
- Extract all explicitly stated data, parameters, variables, objectives, and constraints without omission.
- Do not summarize, truncate, or infer unstated information.
- Represent the problem using structured components:
 - * inputs and parameters
 - * exogenous variables and uncertainties
 - * decision and state variables
 - * objective function
 - * constraints
 - * transition function (if applicable)

Type inference logic:

- Determine variable types using a strict priority:
 - 1) explicit textual indicators,
 - 2) cost unit semantics (per-item vs per-measure),
 - 3) naming semantics (discrete objects vs divisible quantities).
- Do not guess or add qualifiers not stated in the text.

Representation rules:

- Use Python-style symbolic expressions.
- Materialize all tabular or graph data as explicit nested lists.
- Express all constraints as atomic expressions.

Output format (EXACT):

Return exactly one JSON object with the following structure:

```
{
  "problem_description": "...",
  "decision_variables": [
    { "name": "...", "type": "INTEGER/CONTINUOUS/BINARY", "description": "..." }
  ],
  "inputs": [
    { "name": "...", "value": "...", "units": "...", "description": "..." }
  ],
  "exogenous_variables": [],
  "exogenous_uncertainties": [],
  "state_variables": [],
  "transition_function": "",
  "objective_function": {
    "direction": "minimize/maximize",
    "expression": "...",
    "description": "..."
  },
  "constraints": [
    { "expression": "...", "description": "..." }
  ]
}
```

```

    ]
  }

Inputs:
- Problem description: {problem_description}

```

MBR Candidate Re-ranking Prompt

Act as an adjudicator selecting the single best Operations Research formulation from a small set of top candidates with subtle logical differences.

Inputs:

- Problem description: {problem_description}
- Candidate formulations (JSON): {candidates_json}

Decision protocol:

1. Identify functional differences (missing or extra constraints, inequality or objective direction changes, variable type mismatches).
2. Verify against the problem text:
 - Penalize unsupported or omitted constraints.
 - Disqualify candidates using prose instead of symbolic math.
 - Prefer balance equations over redundant static bounds when they implicitly enforce limits.
3. Select the mathematically correct, non-redundant formulation; prefer fewer, more general constraints when multiple candidates are valid.

Output:

Return ONLY a JSON object in the following format:

```

{
  "disagreement_analysis": "Brief summary of key conflicts.",
  "best_candidate_id": 1,
  "confidence": "high|medium|low",
  "reasoning": "Concise justification referencing correctness and avoidance of redundancy."
}

```

Constraints:

- best_candidate_id must match a provided candidate ID.
- No text outside the JSON object.

E.1.2. MATHEMATICAL FORMULATION

Problem Description: Humanitarian Food Distribution Scenario

Imagine you are the director of a non-profit organization tasked with providing food supplies to six regions suffering from a famine. Each region has a certain amount of food already, but they require more to sustain their population through the hardship.

Here are the current quantities of food (in tons) available and the required quantities for each region:

- Region 1 has 42 tons but needs 74 tons.
- Region 2 has 32 tons but needs 476 tons.
- Region 3 has 398 tons but only needs 2 tons.

- Region 4 has 224 tons but needs 235 tons.
- Region 5 has 210 tons but needs 221 tons.
- Region 6 has 209 tons but only needs 72 tons.

You have the ability to transfer food supplies from one region to another. However, the cost of transportation varies depending on which regions you are transferring food between. Below is a list detailing the cost of moving food from one region to any other:

- To move food from/to Region 1: To Region 2 costs 16, to Region 3 costs 48, to Region 4 costs 42, to Region 5 costs 50, to Region 6 costs 8.
- To move food from/to Region 2: To Region 1 costs 27, to Region 3 costs 23, to Region 4 costs 37, to Region 5 costs 39, to Region 6 costs 29.
- To move food from/to Region 3: To Region 1 costs 49, to Region 2 costs 39, to Region 4 costs 33, to Region 5 costs 50, to Region 6 costs 6.
- To move food from/to Region 4: To Region 1 costs 23, to Region 2 costs 49, to Region 3 costs 46, to Region 5 costs 50, to Region 6 costs 6.
- To move food from/to Region 5: To Region 1 costs 45, to Region 2 costs 47, to Region 3 costs 48, to Region 4 costs 26, to Region 6 costs 39.
- To move food from/to Region 6: To Region 1 costs 33, to Region 2 costs 11, to Region 3 costs 9, to Region 4 costs 4, to Region 5 costs 12.

Your mission is to ensure every region receives the food it needs while keeping the transportation cost as low as possible. What would be the minimum cost to make sure all regions have enough food?

Mathematical Formulation

```
{
  "decision_variables": [
    {
      "name": "x[i, j]",
      "type": "CONTINUOUS",
      "description": "tons of food shipped from region i to region j (i != j, i, j in {1, ..., 6})"
    }
  ],
  "inputs": [
    {
      "name": "current_food",
      "value": [42, 32, 398, 224, 210, 209],
      "units": "tons",
      "description": "current quantity of food available in each region i (i = 1, ..., 6)"
    },
    {
      "name": "required_food",
      "value": [74, 476, 2, 235, 221, 72],
      "units": "tons",
      "description": "required quantity of food for each region i (i = 1, ..., 6)"
    }
  ]
}
```

```

    "name": "transportation_costs",
    "value": [
      [0, 16, 48, 42, 50, 8],
      [27, 0, 23, 37, 39, 29],
      [49, 39, 0, 33, 50, 6],
      [23, 49, 46, 0, 50, 6],
      [45, 47, 48, 26, 0, 39],
      [33, 11, 9, 4, 12, 0]
    ],
    "units": "cost units per ton",
    "description": "matrix where entry [i][j] is the cost of transporting one ton
of food from region i+1 to region j+1"
  },
  "exogenous_variables": [],
  "exogenous_uncertainties": [],
  "state_variables": [],
  "transition_function": "",
  "objective_function": {
    "direction": "minimize",
    "expression": "sum(transportation_costs[i][j] * x[i,j] for i in regions for j in
regions if i != j)",
    "description": "minimize total transportation cost of shipping food between
regions"
  },
  "constraints": [
    {
      "expression": "current_food[i] + sum(x[j,i] for j in regions if j != i) -
sum(x[i,j] for j in regions if j != i) >= required_food[i] for all i in regions",
      "description": "each region must end up with at least its required amount of
food"
    },
    {
      "expression": "x[i,j] >= 0 for all i, j in regions with i != j",
      "description": "non-negativity of shipped quantities"
    }
  ]
}

```

E.1.3. COMPONENT MBR RE-RANKING

In this stage, the system employs the two-step refinement process illustrated in Figure 4. First, it generates 5 candidate extractions and filters them using component-level consensus scores (Stage 1). Second, an LLM-based reranker analyzes the top 3 candidates to select the final output based on logical completeness (Stage 2).

Stage 1: Component-Level MBR Filtering

```

{
  "total_candidates": 5,
  "filtering_logic": "Select Top-3 based on embedding consensus score",
  "scores": [
    { "id": 5, "score": 0.861, "status": "PASSED" },
    { "id": 2, "score": 0.860, "status": "PASSED" },
    { "id": 3, "score": 0.857, "status": "PASSED" },
    { "id": 4, "score": 0.785, "status": "FILTERED (Score too low)" },
    { "id": 1, "score": 0.776, "status": "FILTERED (Score too low)" }
  ]
}

```

Stage 2: LLM Re-ranking (Selection from Top 3)

```
{
  "disagreement_analysis": "Main difference is the balance constraint. Candidates 2
    and 3 impose strict equality: final stock must equal the exact requirement,
    expressed as (out-in) = current - required. Because total initial supply (1115)
    exceeds total need (1080) by 35 tons, this global equality system is infeasible
    (sum of left sides is 0, right side 35). Candidate 5 instead uses an inequality:
    current + in - out >= required, which matches the text ('make sure all regions
    have enough food') and allows leftover surplus. All candidates include
    non-negativity and the correct cost-minimisation objective; Candidate 2 also
    allows x[i][i] flows and omits the i!=j restriction. No other material
    disagreements.",
  "best_candidate_id": 5,
  "confidence": "high",
  "reasoning": "Candidate 5 is the only formulation that faithfully captures the
    requirement 'at least the needed amount' with a >= constraint, therefore remains
    feasible given the overall surplus. It omits self-ship variables, has the correct
    minimisation objective and enforces non-negativity. Candidates 2 and 3, by forcing
    exact equality, would make the model infeasible for the stated data, so they must
    be rejected."
}
```

Constraint Comparison: Selected vs. Rejected

```
{
  "Selected Formulation": {
    "constraints": [
      {
        "expression": "current_food[i] + sum(x[j,i] for j in regions if j != i) -
          sum(x[i,j] for j in regions if j != i) >= required_food[i] for all i in regions",
        "description": "each region must end up with at least its required amount of
          food (allows surplus)"
      }
    ]
  },
  "Rejected Formulation": {
    "constraints": [
      {
        "expression": "sum(x[i][j] for j in regions) - sum(x[j][i] for j in regions)
          == current_food[i] - required_food[i] for all i in regions",
        "description": "food balance for each region: net shipments out minus net
          shipments in equals its initial surplus or deficit (forces exact equality, causing
          infeasibility)"
      }
    ]
  }
}
```

E.2. Solver Recommender**Solver Recommender Prompt**

Inputs:

- A decision problem specified as KEY_INGREDIENTS: {key_ingredients}
- A candidate solver list SOLVERS: {solvers}

Goal:

Recommend a ranked list of solvers to solve the given problem.

System logic:

- Assume a working Gurobi commercial license is available, and mention this explicitly in the recommendation.
- Do not recommend open-source alternatives solely for licensing reasons, since Gurobi can be used without restrictions.
- Rank solvers primarily by:
 - 1) simplicity of use and integration,
 - 2) expected suitability for the problem size and complexity implied by KEY_INGREDIENTS.
- Provide recommendations only (no implementation details or code beyond what is required to communicate solver choice rationale).

Solver Ranking

Recommended solvers for this LP-type minimum-cost transportation problem (ranked from most convenient to least):

 1. gurobipy (Gurobi's Python API) [SELECTED]

Why:

- Native linear-programming engine; solves this six-node balanced transportation model in milliseconds.
- Extremely simple modeling interface (addVars, addConstrs, setObjective).
- You already have a commercial license and the environment variables GRB_WLSACCESSID, GRB_WLSSECRET, and GRB_LICENSEID are set, so you can start immediately.

How to install:

```
pip install gurobipy
```

How to activate license:

```
import os, gurobipy as gp
options = {
    'WLSACCESSID': os.environ.get('GRB_WLSACCESSID'),
    'WLSSECRET':   os.environ.get('GRB_WLSSECRET'),
    'LICENSEID':   int(os.environ.get('GRB_LICENSEID'))
}
env = gp.Env(params=options)
```

 2. CVXPY (open-source, high-level convex modeling)

Why:

- Very concise (problem can be written in ~15 lines).
- Automatically picks the best open-source LP solver installed (default ECOS/CBC).

 3. OR-Tools (Google)

Why:

- Contains a dedicated min-cost flow interface even higher-level than plain LP.
- Pure Python wheel, no external solver needed.

```
-----
4. Pyomo / 5. SciPy / 6. pycscipopt
-----
```

(Listed as alternatives but ranked lower due to verbosity or unnecessary overhead for this specific small LP.)

```
=====
Summary
=====
```

- Use gurobipy firstfastest path to an optimal answer, license already configured.
- CVXPY and OR-Tools are the simplest open-source fall-backs.

E.3. Simulator

As depicted in the `simulator_code/` branch of Figure 7, the coding agent first materializes a simulation environment. Generated via a single prompt provided below, this module acts as the independent validator for any proposed solutions.

The core logic resides in `constraints.py`, which enforces the physical rules of the system (e.g., flow conservation and non-negativity). A snippet of this generated verification logic is provided below.

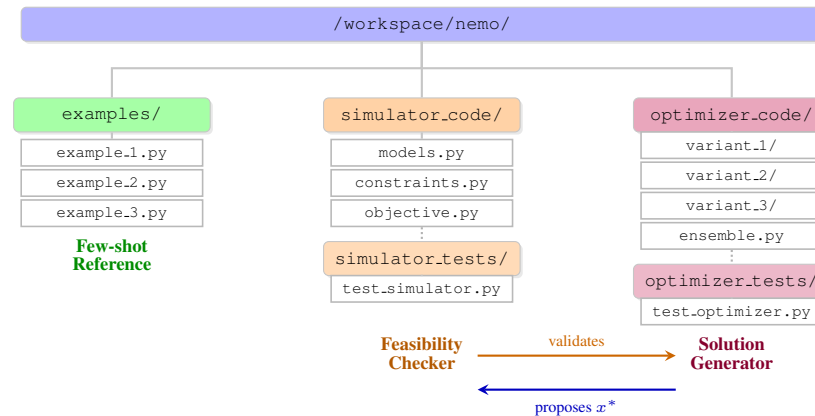


Figure 7. Coding agent workspace structure. Retrieved examples are materialized as executable Python files in `examples/`. The agent independently generates a simulator and an optimizer; the optimizer contains three independent implementations of the optimizer. Associated test suites in `simulator_tests/` and `optimizer_tests/` provide regression, feasibility, and solver-consistency checks.

Simulator Creation Prompt

Define a simulator (not an optimizer) for a decision process specified by a structured `KEY_INGREDIENTS` JSON, including the full natural-language `problem_description`.

System logic:

- The simulator represents the mechanics of the decision process exactly as specified in `KEY_INGREDIENTS`.
- It provides explicit representations for:
 - decision variables
 - system state
 - exogenous inputs
 - uncertain variables
- Given a concrete set of inputs, the simulator:
 - 1) checks all constraints and reports violations,
 - 2) evaluates the objective and related metrics only when constraints are satisfied,
 - 3) applies the transition function to update system state when applicable.
- The simulator evaluates provided decisions only and does not perform optimization,

```

    search, or inference of optimal actions.
- The simulator must not assume optimality, solver behavior, or hidden objectives
  beyond what is stated.

Architectural separation:
- The simulator is a standalone system component and is logically isolated from any
  optimizer or solver logic.
- It serves as a ground-truth evaluator for decisions, not a decision generator.

Directory structure (architectural contract):
/workspace/nemo/
|-- simulator_code/           (simulator logic)
|-- simulator_tests/         (validation and diagnostics)

Inputs:
- Decision process specification: {key_ingredients}

```

Simulator Implementation (constraints.py)

```

def check_all_constraints(decision, inputs):
    violations = []

    # 1. Check Non-negativity
    if (v := check_non_negativity_constraint(decision)):
        violations.append(v)

    # 2. Check Food Requirements (Flow Balance)
    violations.extend(check_food_requirement_constraint(decision, inputs))

    return violations

def check_food_requirement_constraint(decision, inputs):
    """
    Check that each region ends up with at least its required amount of food.
    Logic: Final = Initial + Inflow - Outflow >= Required
    """
    violations = []

    # Start with current food (copy to avoid mutating input)
    net_food = inputs.current_food.copy()

    # Apply all shipments
    for (from_region, to_region), amount in decision.shipments.items():
        net_food[from_region - 1] -= amount # Outflow
        net_food[to_region - 1] += amount   # Inflow

    # Check if each region has enough food
    for region in range(1, 7):
        idx = region - 1
        if net_food[idx] < inputs.required_food[idx]:
            violations.append(ConstraintViolation(
                constraint_name="food_requirement",
                description=f"Region {region} insufficient food",
                details=f"Has {net_food[idx]:.2f} tons, needs
{inputs.required_food[idx]}")
            ))

    return violations

```

E.4. Optimizer

E.4.1. PROMPTS

Optimizer Creator Prompt

Solve a single optimization problem defined by KEY_INGREDIENTS and produce a valid solver-derived result.

Core logic:

- Interpret the problem strictly and literally from KEY_INGREDIENTS, including the full problem_description.
- Build an optimization model that exactly matches the specified decision variables, constraints, and objective.
- Do not introduce additional constraints, objectives, assumptions, or problem modifications.
- Solve the problem using one of the provided LIBRARY/SOLVER options.
- If multiple solvers are provided, attempt them in ranked order until a valid result is obtained.
- Results must be produced by an actual solver run; fabricated or assumed solutions are forbidden.

System invariants:

- The simulator is treated as immutable and must not be modified or relied upon during optimization.
- The optimizer solves only the given problem instance (no toy data or alternate inputs).
- Solver status must be reported truthfully (optimal, infeasible, unbounded, time_limit, or error).

Directory structure (logical contract):

The optimizer exists as a separate module alongside the simulator, with optimizer logic isolated from simulator logic:

```
/workspace/nemo/
|-- simulator_code/      (immutable; not used by optimizer)
|-- simulator_tests/     (immutable)
|-- optimizer_code/      (optimizer implementation)
|-- optimizer_tests/     (optimizer outputs and validation artifacts)
```

Output format (EXACT):

Produce a JSON object with the following structure:

```
{
  "optimal_variables": {"var_name": value, ...},
  "optimal_objective_value": float,
  "status": "optimal|infeasible|unbounded|time_limit|error",
  "solver_info": {
    "solver_name": "string",
    "solve_time": float,
    "iterations": int,
    "gap": float
  }
}
```

Inputs:

- Optimization problem: {key_ingredients}
- Solver recommendations: {library_recommendation}

Optimizer Self-Consistency Prompt

Solve a single optimization problem using an ensemble of {num_variants} independent optimizer variants and produce a consensus solution.

ENSEMBLE LOGIC:

- Create {num_variants} independent optimizer variants.
- All variants solve the SAME problem defined by KEY_INGREDIENTS.
- Variants are independent implementations to reduce modeling and implementation errors.
- Each variant produces its own optimization result.
- Final solution is obtained via majority voting across variants.

CONSENSUS RULES:**Status consensus:**

- Count solver statuses across variants.
- Select the most frequent status.
- Break ties using the priority order:
optimal > time_limit > infeasible > unbounded > error

Objective value consensus:

- Group objective values that agree within tolerance:
 - * relative tolerance: 1e-6
 - * absolute tolerance: 1e-9
- Select the objective value with the largest agreement group.
- If tied, select the median of the tied group.

Variable consensus:

- Identify variants that produced the majority objective value.
- Select decision variables from one agreeing variant.
- Do NOT average integer or binary variables.

Tracking:

- Track successful vs failed variants.
- Track solver agreement and objective agreement.
- Record variants that produced outlier or failed results.

Variant output format (EXACT):

Each variant outputs:

```
{
  "optimal_variables": {"var_name": value, ...},
  "optimal_objective_value": float,
  "status": "optimal|infeasible|unbounded|time_limit|error",
  "solver_info": {
    "solver_name": "string",
    "solve_time": float,
    "iterations": int,
    "gap": float
  }
}
```

FINAL ENSEMBLE OUTPUT FORMAT (EXACT):

The ensemble produces a consensus result:

```
{
  "optimal_variables": {"var_name": consensus_value, ...},
  "optimal_objective_value": majority_objective_value,
  "status": "majority_vote_status",
  "solver_info": {
    "ensemble_size": {num_variants},
    "solvers_used": ["solver names"],
```

```

    "consensus_solvers": ["solvers agreeing"]
  },
  "consensus_info": {
    "num_variants": int,
    "num_successful": int,
    "num_failed": int,
    "status_distribution": {},
    "solver_agreement": int,
    "objective_agreement": int,
    "objective_agreement_ratio": float,
    "num_unique_objectives": int,
    "failed_variants": []
  },
  "variant_results": [
    {
      "variant_name": "variant_1",
      "solver": "solver_name",
      "status": "optimal",
      "objective_value": float,
      "solve_time": float
    }
  ]
}

```

Asymmetric Validation Prompt

Validate optimizer results against the simulator. Modify optimizer code and re-run only if validation fails.

Rules:

- No mocked solutions or manual edits to optimization_results.json.
- If re-running is needed, execute the real optimizer script.
- Validation code must live under optimizer_tests/.

Process:

1. Load optimizer_tests/optimization_results.json.
2. Validate results using the simulator (read-only).
3. If valid, exit. If invalid, fix code, re-run optimizer, and re-validate (max 3 iterations).

Checks:

- Verify optimal_variables and dimensions.
- Validate constraints and bounds via the simulator.
- Recompute objective and compare (tolerance 1e-4).
- Flag trivial all-zero optimal solutions.
- Identify responsible optimizer files and required fixes.

Output format (EXACT):

Write optimizer_tests/validation_results.json as:

```

{
  "passed": bool,
  "num_validation_iterations": int,
  "problem_analysis": {
    "problem_feasible": bool,
    "has_trivial_solutions": bool,
    "infeasibility_analysis": {}
  }
},

```

```

    "input_verification": {},
    "constraint_violations": [],
    "bound_violations": [],
    "objective_verification": {
        "optimizer_value": float,
        "simulator_value": float,
        "difference": float,
        "match": bool
    },
    "validation_history": [
        {
            "iteration": int,
            "passed": bool,
            "issues_found": [],
            "fixes_applied": []
        }
    ]
}

```

E.4.2. GENERATED OPTIMIZER CODE

Moving to the `optimizer_code/` branch of Figure 7, the coding agent generates three independent solver implementations (Variants 1–3) to enable self-consistency checking. Below we present the code for Variant 1, which utilizes the Gurobi solver as recommended.

Additionally, we capture the full agent interaction history using the *OpenHands* Trajectory API. This allows us to trace the agent’s debugging steps—specifically how it corrects syntax errors or formulation bugs—providing a granular audit trail for the system’s reasoning process.

Optimizer Implementation (Variant 1 - Gurobi)

```

# --- optimizer.py (Main Driver) ---
class FoodDistributionOptimizer:
    def build_model(self) -> None:
        # 1. Create Model
        self.model = gp.Model("FoodDistribution", env=self.env)

        # 2. Create Variables (x[i,j])
        self.x_vars = {}
        for i in range(1, self.num_regions + 1):
            for j in range(1, self.num_regions + 1):
                if i != j: # No shipments to self
                    self.x_vars[(i, j)] = self.model.addVar(
                        lb=0.0, # Non-negativity constraint
                        name=f"x_{i}_{j}"
                    )

        # 3. Add Constraints & Objective
        add_all_constraints(self.model, self.x_vars, self.inputs)
        set_objective_function(self.model, self.x_vars, self.inputs)

        self.model.update()

    def solve(self, time_limit_seconds=None):
        if self.model is None:
            self.build_model()
        if time_limit_seconds:
            self.model.setParam('TimeLimit', time_limit_seconds)

        self.model.optimize()

```

```

        return self._process_results(self.model.Runtime)

# --- constraints.py (Mathematical Logic) ---
def add_food_requirement_constraints(model, x_vars, inputs):
    """
    Enforces: current_food[i] + Inflow - Outflow >= required_food[i]
    """
    num_regions = len(inputs.current_food)

    for i in range(num_regions):
        region = i + 1 # 1-based indexing

        # Calculate incoming: sum(x[j,i] for j != i)
        incoming = gp.quicksum(
            x_vars.get((j+1, region), 0)
            for j in range(num_regions) if j+1 != region
        )

        # Calculate outgoing: sum(x[i,j] for j != i)
        outgoing = gp.quicksum(
            x_vars.get((region, j+1), 0)
            for j in range(num_regions) if j+1 != region
        )

        # Add constraint (ALLOWS surplus food)
        model.addConstr(
            inputs.current_food[i] + incoming - outgoing >= inputs.required_food[i],
            name=f"food_requirement_region_{region}"
        )

```

E.4.3. OPTIMIZER RESULTS & VALIDATION

Following code generation, the system executes the `ensemble.py` script (see Figure 7). This orchestrator triggers all three generated solver variants in parallel and aggregates their results to verify mathematical consensus.

The output below shows the exact JSON structure returned by this ensemble execution, confirming that the agent correctly formatted the response and achieved unanimous agreement on the objective value.

Optimizer Results

```

{
  "optimal_variables": {
    "0,1": 0.0, "0,2": 0.0, "0,3": 0.0, "0,4": 0.0, "0,5": 0.0,
    "2,5": 361.0, // ... (Non-zero active route)
    "3,0": 32.0, // ... (Non-zero active route)
    "5,1": 444.0, // ... (Non-zero active route)
    "5,3": 43.0, // ... (Non-zero active route)
    "5,4": 11.0 // ... (Non-zero active route)
    // ... (Remaining zero-flow variables omitted for brevity)
  },
  "optimal_objective_value": 8090.0,
  "status": "optimal",
  "solver_info": {
    "ensemble_size": 3,
    "solvers_used": [ "Gurobi", "CVXPY (ECOS)", "OR-Tools (GLOP)" ],
    "consensus_solvers": [ "Gurobi", "CVXPY (ECOS)", "OR-Tools (GLOP)" ]
  },
  "consensus_info": {
    "num_variants": 3,
    "num_successful": 3,
    "num_failed": 0,

```

```

    "status_distribution": { "optimal": 3 },
    "solver_agreement": 3,
    "objective_agreement": 3,
    "objective_agreement_ratio": 1.0,
    "num_unique_objectives": 1,
    "failed_variants": []
  },
  "variant_results": [
    {
      "variant_name": "variant_1",
      "solver": "Gurobi",
      "status": "optimal",
      "objective_value": 8090.0,
      "solve_time": 0.0019
    },
    {
      "variant_name": "variant_2",
      "solver": "CVXPY (ECOS)",
      "status": "optimal",
      "objective_value": 8090.00000015,
      "solve_time": 0.0359
    },
    {
      "variant_name": "variant_3",
      "solver": "OR-Tools (GLOP)",
      "status": "optimal",
      "objective_value": 8090.0,
      "solve_time": 0.0016
    }
  ]
}

```

Finally, the system validates the proposed solution against the simulator. As shown in the validation log below, the simulator independently verifies that the solution satisfies all constraints (returns an empty violation list) and that the re-calculated objective value matches the optimizer's report exactly.

Validation Results

```

{
  "passed": true,
  "validation_history": [
    {
      "iteration": 0,
      "passed": true,
      "issues_found": [],
      "fixes_applied": []
    }
  ],
  "problem_analysis": {
    "problem_feasible": true,
    "has_trivial_solutions": false
  },
  "constraint_violations": [], // Empty list confirms physical feasibility
  "objective_verification": {
    "passed": true,
    "optimizer_value": 8090.0,
    "simulator_value": 8090.0,
    "difference": 0.0, // Perfect alignment with the simulator
    "issues": []
  }
}

```

```

}
}

```

E.4.4. RETRIEVED FEW-SHOT EXAMPLES

As a preliminary step before code generation, the system retrieves relevant solved instances from the vectorstore based on semantic similarity. These samples are uploaded directly into the *OpenHands* workspace (specifically the `examples/` directory shown in Figure 7), providing the agent with concrete reference implementations. Below is one such retrieved artifact.

Retrieved Code Artifact

```

"""
Training Example 1 | Similarity: 0.716 | Type: transportation
Question: Managing Food Distribution Across Six Cities
... (Problem Description: surplus/deficits and shipment costs) ...
"""

import gurobipy as gp
from gurobipy import GRB

# Data
cities = [0, 1, 2, 3, 4, 5]
supply = {0: 17, 1: 5, 2: 3, 3: -7, 4: -21, 5: 3}
cost = {
    (0, 1): 3, (0, 2): 6, (0, 3): 4, (0, 4): 8, (0, 5): 5,
    # ... (truncated for brevity)
}
capacity = {
    (0, 1): 32, (0, 2): 16, (0, 3): 38, (0, 4): 14, (0, 5): 34,
    # ... (truncated for brevity)
}

# Model
model = gp.Model("FoodDistribution")
x = model.addVars(cities, cities, name="x")

# Objective
model.setObjective(gp.quicksum(cost[i,j]*x[i,j] for i,j in cost), GRB.MINIMIZE)

# Constraints
for i in cities:
    # Supply/Demand Balance
    model.addConstr(
        gp.quicksum(x[i,j] for j in cities) -
        gp.quicksum(x[j,i] for j in cities) == supply[i]
    )

for i,j in capacity:
    model.addConstr(x[i,j] <= capacity[i,j])

model.optimize()

```